6.0 VU Theoretische Informatik (192.017)
Teil 3: Formale Sprachen

Marion OSWALD

Wintersemester 2026

Teil 3: Formale Sprachen

Inhalt

- Grundlagen (Wiederholung)
- Turingmaschinen (revisited)
- Reguläre Sprachen (Wiederholung)
- Grammatiken
- Kontextfreie Grammatiken und Sprachen
- Jenseits der Kontextfreiheit
- Chomsky Hierarchie

Grundlagen: Wörter

Alphabet: *endliche*, nicht-leere Menge atomarer Symbole (Σ, T)

Wort über Σ: endliche Folge von Symbolen aus Σ

Länge eines Wortes w über Σ (geschrieben $|w|$): Anzahl der Symbole, die w enthält

Leerwort: Wort mit der Länge 0, geschrieben ε, d.h. $|\varepsilon| = 0$

Für ein Wort w über Σ und ein Symbol $a \in \Sigma$ bezeichnen wir die **Anzahl der Symbole** a **in** w mit $|w|_a$.

Konkatenation: Hintereinanderschreiben von Wörtern
Seien x, y Wörter mit $|x| = n$, $|y| = m$, dann ist $x \cdot y = xy$ und $|xy| = n + m$

Potenzbildung: Verkettung eines Wortes w mit sich selbst, wobei $w^0 = \varepsilon$ und $w^n = ww^{n-1}$

Sei $w = a_1 a_2 \dots a_{n-1} a_n$, dann ist $w^r = a_n a_{n-1} \dots a_2 a_1$ das **Spiegelbild** von w.

Ein Wort w heißt **Palindrom**, wenn $w = w^r$ gilt.

Grundlagen: (Formale) Sprachen

Σ^+: Menge aller (nicht-leeren) Wörter über Σ
Σ^*: Menge aller Wörter (inklusive ε) über Σ

Formale Sprache: beliebige Teilmenge L von Σ^*, $L \subseteq \Sigma^*$

$(\Sigma^*, \cdot, \varepsilon)$ bildet ein *Monoid*.

Beispiele für Sprachen

- $\{\}$
- $\{\varepsilon\}$
- $\{\varepsilon, ab, abab, ababab, ...\} = \{(ab)^n \mid n \geq 0\} \quad \Sigma = \{a, b\}$
- $\{\varepsilon, ab, aabb, aaabbb, ...\} = \{a^n b^n \mid n \geq 0\} \quad \Sigma = \{a, b\}$
- $\{w \in \Sigma^* \mid w = w^r\} \quad \Sigma = \{0, 1\}$
- $\{w \in \{a, b, c\}^* \mid |w|_a = |w|_b = |w|_c\}$

$\mathcal{P}(\Sigma^*)$: **Menge aller Sprachen** $L \subseteq \Sigma^*$

Operationen auf Sprachen

Konkatenation: $L_1 \cdot L_2 = \{w_1 \cdot w_2 \mid w_1 \in L_1, w_2 \in L_2\}$

Bsp: $\{ab, b\}\{a, bb\} = \{aba, abbb, ba, bbb\}$
(Achtung: $\{ab, b\} \times \{a, bb\} = \{(ab, a), (ab, bb), (b, a), (b, bb)\}$)

Potenzbildung: $L^n = \{w_1 ... w_n \mid w_1, ..., w_n \in L\}$
($L^0 = \{\varepsilon\}$ und $L^{n+1} = LL^n$ für $n \geq 0$)

Bsp: $\{ab, ba\}^2 = \{abab, abba, baab, baba\}$

Kleene-Stern[1]: $L^* = \bigcup_{n \geq 0} L^n$ $\qquad$ $(L^+ = \bigcup_{n \geq 1} L^n)$

Bsp: $\{01\}^* = \{\varepsilon, 01, 0101, 010101, ...\} \neq \{0, 1\}^*$

$(\mathcal{P}(\Sigma^*), \cdot, \{\varepsilon\})$ bildet ein *Monoid*.

- $\varepsilon \in L^*$ gilt für alle Sprachen L (z.B.: $\{\}^* = \{\varepsilon\}$)
- $\varepsilon \in L^+$ gilt für eine Sprache L genau dann, wenn $\varepsilon \in L$

[1] benannt nach Stephen Cole Kleene (1909 - 1994)

Rechenregeln für Sprachoperatoren

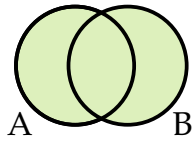
$$
\begin{array}{lcll}
A \cdot (B \cdot C) & = & (A \cdot B) \cdot C & \text{Assoziativität von} \cdot \\
A \cdot (B \cup C) & = & A \cdot B \cup A \cdot C & \text{Distributivität von} \cdot \\
(B \cup C) \cdot A & = & B \cdot A \cup C \cdot A & \text{Distributivität von} \cdot
\end{array}
$$

$$
\begin{array}{rcl}
(A \cup \{\varepsilon\})^* & = & A^* \\
(A^*)^* & = & A^* \\
A \cdot A^* & = & A^+ \\
A^* \cdot A & = & A^+ \\
A^+ \cup \{\varepsilon\} & = & A^*
\end{array}
\qquad
\begin{array}{rcl}
\{\varepsilon\} \cdot A & = & A \\
A \cdot \{\varepsilon\} & = & A \\
\{\} \cdot A & = & \{\} \\
A \cdot \{\} & = & \{\}
\end{array}
$$

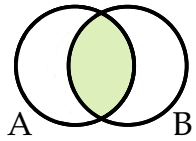
$(\mathcal{P}(\Sigma^*), \cup, \cdot, \{\}, \{\varepsilon\})$ bildet einen *nichtkommutativen Semiring*.

Mengenoperationen auf Sprachen

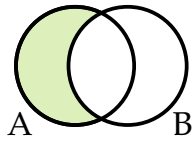
Vereinigung:
$A \cup B = \{x \in \Sigma^* \mid x \in A \text{ oder } x \in B\}$



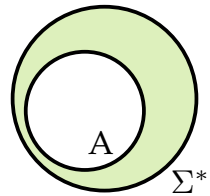
Durchschnitt:
$A \cap B = \{x \in \Sigma^* \mid x \in A \text{ und } x \in B\}$
(A und B sind **disjunkt** wenn $A \cap B = \{\}$)



Differenz:
$A - B = \{x \in \Sigma^* \mid x \in A \text{ und } x \notin B\}$



Komplement:
$\overline{A} = \Sigma^* - A = \{x \in \Sigma^* \mid x \notin A\}$



$A \subseteq B$: A ist eine **Teilmenge** von B (jedes Element von A ist auch ein Element von B)

$A \subset B$: A ist eine **echte Teilmenge** von B (A ist eine Teilmenge von B, die nicht gleich B ist)

$A = B$ wenn $A \subseteq B$ und $B \subseteq A$

Abgeschlossenheit

Sind eine Menge B und Operatoren auf Teilmengen von B gegeben, so kann man sich natürlich fragen, ob die Anwendung der Operatoren auf diese Teilmengen von B wiederum Teilmengen von B ergibt.

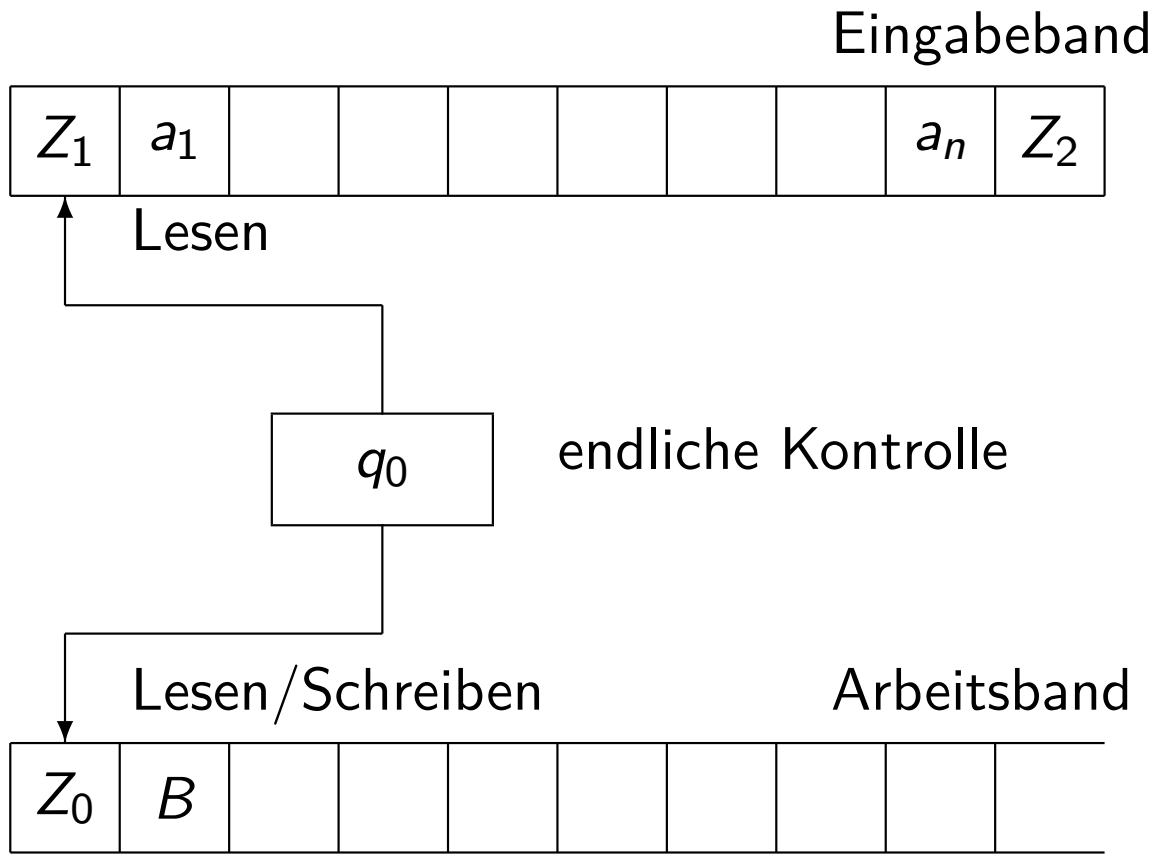
Sei B eine Menge und $f : B^n \longrightarrow B$ eine Funktion. Eine Menge $A \subseteq B$ heißt **abgeschlossen** unter f, wenn gilt:

$$x_1, ..., x_n \in A \Rightarrow f(x_1, ..., x_n) \in A$$

Beispiel

Die Menge der natürlichen Zahlen $\mathbb{N}$ ist abgeschlossen unter Addition, aber nicht unter Subtraktion.

Turingmaschinen[2] revisited



[2] Alan M. Turing (1912 - 1954)

Turingmaschinen

Eine Turingmaschine M besteht aus folgenden Komponenten:

- Das *Eingabeband* kann von links nach rechts gelesen werden. Es beinhaltet eine endliche Folge von Zeichen (das Eingabewort), wobei das Eingabewort vom Anfangssymbol Z_1 und vom Endsymbol Z_2 begrenzt ist.
- Das *Arbeitsband* kann beliebig gelesen und beschrieben werden.
- Die *endliche Kontrolle* kann einen Zustand aus einer endlichen Menge von Zuständen annehmen und steuert den Lesekopf auf dem Eingabeband und den Lese-/Schreibkopf auf dem Arbeitsband; die Kopfbewegungen sind L, R, S (links/left, rechts/right, stehenbleiben/stay).

Formale Beschreibung einer Turingmaschine

Eine *Turingmaschine* ist ein Achttupel

$$M = (Q, T, \Gamma, \delta, q_0, \{Z_0, Z_1, Z_2\}, B, F),$$

- Q endliche Menge von Zuständen,
- T Alphabet des Eingabebandes,
- Γ Alphabet des Arbeitsbandes,
- δ Übergangsfunktion,
- $q_0 \in Q$ Startzustand,
- $Z_0 \in \Gamma$ linkes Begrenzungssymbol auf Arbeitsband, Z_1, Z_2 Begrenzungssymbole des Eingabewortes,
- $B \in \Gamma$ Blanksymbol,
- $F \subseteq Q$ Menge von Endzuständen.

Übergänge

$$(q, a, X; p, Y, D_E, D_A) \in \delta$$

M liest im Zustand q auf dem Eingabeband Symbol a und auf Arbeitsband Symbol X

- wechselt nun davon abhängig in den Zustand p,
- überschreibt auf dem Arbeitsband Symbol X durch Symbol Y ,
- bewegt auf dem Eingabeband den Lesekopf in die durch D_E beschriebene Richtung und auf dem Arbeitsband den Lese-/Schreibkopf in die durch D_A beschriebene Richtung (D_E, $D_A \in \{L, R, S\}$).

(Deterministische) Turingmaschinen: akzeptierte Sprache

Akzeptierte Sprache

Die von der Turingmaschine M *akzeptierte Sprache* $L(M)$ besteht aus genau all jenen Wörtern, bei deren Anaylse M einen Endzustand erreicht.

Deterministische Turingmaschine

Eine Turingmaschine M heißt *deterministisch*, wenn für alle $(q, a, X) \in Q \times V_T \times \Gamma$ höchstens ein Element $(q, a, X; p, Y, D_E, D_A) \in \delta$ existiert; wir schreiben dann auch $\delta(q, a, X) = (p, Y, D_E, D_A)$.

Turingmaschinen - Beispiel

Turingmaschine, die $\{a^n b^n c^n \mid n \geq 1\}$ akzeptiert

$M = (\{p, q, r, s\}, \{a, b, c\}, \{Z_0, A, B, C\}, \delta, p, \{Z_0, Z_1, Z_2\}, B, \{s\})$

wobei

$$
\begin{aligned}
1: &\quad \delta(p, a, B) = (p, A, R, R) \\
2: &\quad \delta(p, b, B) = (q, B, S, L) \\
3: &\quad \delta(q, b, A) = (q, C, R, L) \\
4: &\quad \delta(q, c, Z_0) = (r, Z_0, S, R) \\
5: &\quad \delta(r, c, C) = (r, B, R, R) \\
6: &\quad \delta(r, Z_2, B) = (s, B, S, L)
\end{aligned}
$$

Turingmaschinen - Beispiel ctd.

Dabei passiert Folgendes:
1 : für jedes eingelesenes Symbol a wird ein A aufs Arbeitsband geschrieben;

2 : wenn das erste Symbol b eingelesen wird, geht M einen Schritt nach links; dieses erste Symbol b wird bei der ersten Anwendung von Übergang 3 nochmals gelesen;

3 : für ein eingelesenes Symbol b wird ein A auf dem Arbeitsband mit C überschrieben;

4 : wird das erste Symbol c gelesen, so muss M genau Z_0 erreicht haben;

5 : für jedes eingelesene Symbol c wird nun ein C auf dem Arbeitsband gelöscht (durch das Blanksymbol B ersetzt);

6 : wird mit dem Ende der Eingabe (d.h., dem Erreichen von Z_2) gleichzeitig das Ende der Symbole C auf dem Arbeitsband erreicht, so hält die Turingmaschine M im Endzustand s, und hat damit die Eingabe akzeptiert.

Normalform

Eine (deterministische) Turingmaschine ist in **Normalform**, wenn

- sie nur einen Endzustand besitzt,
- das Arbeitsband am Ende eines akzeptierenden Laufes der Turingmaschine leer ist und
- der letzte Übergang von der Gestalt $\delta(s, Z_2, Z_0) = (f, Z_0, S, R)$ ist, wobei f dieser einzige Endzustand und s ein beliebiger anderer Zustand ist.

Normalform: Beispiel

M' akzeptiert $\{a^n b^n c^n \mid n \geq 1\}$ in Normalform

$M' = (\{p, q, r, s, f\}, \{a, b, c\}, \{Z_0, A, B, C\}, \delta, p, \{Z_0, Z_1, Z_2\}, B, \{f\})$
wobei

Übergänge 1 bis 6 wie bei M aus vorigem Beispiel.

Damit M' jedoch mit leerem Band akzeptieren kann, benötigen wir zusätzlich noch folgende Übergänge:

$7 : \delta(s, Z_2, B) = (s, B, S, L)$ sowie
$8 : \delta(s, Z_2, Z_0) = (f, Z_0, S, R)$.

TM: eingeschränkte Varianten - LBA

Ein **linear beschränkter Automat** (LBA) ist eine Turingmaschine, die auf dem Arbeitsband nur höchstens soviel Platz verwendet wie das Eingabewort lang ist (der Platz darf sogar eine lineare Funktion der Länge des Eingabewortes sein).

LBA

Eine Turingmaschine M heißt *linear beschränkter Automat*, wenn es eine lineare Funktion $cn + d$ so gibt, dass die Turingmaschine M für jedes Eingabewort w der Länge n während der Analyse höchstens $cn + d$ Felder auf dem Arbeitsband benötigt.

TM: Eingeschränkte Varianten - Kellerautomaten

Turingmaschine erfüllt **Kellerautomatenbedingung**, wenn sie

- auf dem Eingabeband nie nach links gehen kann und
- für den Lese-/Schreibkopf auf dem Arbeitsband in jeder Konfiguration gilt, dass
  - links davon nur Nicht-Blanksymbole und
  - rechts davon nur Blanksymbole stehen können.

Im Wesentlichen ist also ein Kellerautomat eine Turingmaschine, die das Arbeitsband als sogenannten Keller verwendet, d.h., der Kellerautomat darf in jedem Schritt nur das oberste Symbol lesen und dieses dann löschen bzw. durch ein Symbol über dem sogenannten Kelleralphabet ersetzen.

Ein Kellerautomat funktioniert wie ein Stack – nach dem LIFO – Last-in-first-out-Prinzip.

Kellerautomaten: akzeptierte Sprache

Es gibt zwei Möglichkeiten, die vom Kellerautomaten akzeptierte Sprache zu definieren:

1. Kellerautomat befindet sich im Sinne einer Turingmaschine nach dem Einlesen des gesamten Eingabewortes in einem akzeptierenden Zustand.
2. Kellerautomat akzeptiert durch leeren Keller.

Normalform: Eingabewort wird durch Endzustand und leeren Keller akzeptiert d.h., beim Übergang in den (einzigen) Endzustand wird das rechte Begrenzungssymbol Z_2 des Eingabebandes und im Keller gleichzeitig das Kellergrundsymbol gelesen. (entspricht der Normalform für Turingmaschinen)

Kellerautomaten: Beispiel

Kellerautomat M in Normalform an, der $L = \{0^n 1^{2n+1} \mid n \geq 0\}$ akzeptiert

$$M = (\{q_0, q_1, q_2, q_f\}, \{0, 1\}, \{Z_0, A\}, \delta, q_0, \{Z_0, Z_1, Z_2\}, \{q_f\}),$$

wobei

$$\begin{aligned}
1: &\quad \delta(q_0, 0, B) = \{(q_1, A, S, R)\} \\
2: &\quad \delta(q_1, 0, B) = \{(q_0, A, R, R)\} \\
3: &\quad \delta(q_0, 1, B) = \{(q_2, B, R, L)\} \\
4: &\quad \delta(q_2, 1, A) = \{(q_2, B, R, L)\} \\
5: &\quad \delta(q_2, Z_2, Z_0) = \{(q_f, Z_0, S, R)\}
\end{aligned}$$

Kellerautomaten: Beispiel ctd.

Dabei passiert Folgendes:
$1, 2$: für jedes eingelesenes Symbol 0 werden 2 Symbole A in den Keller geschrieben;
3 : das erste Symbol 1 wird überlesen;
4: nun wird für jedes eingelesene Symbol 1 ein Symbol A im Keller gelöscht;
5 : wird gleichzeitig mit dem Ende der Eingabe (dem Erreichen von Z_2) das Kellergrundsymbol Z_0 erreicht, geht der Kellerautomat in den Enzustand q_f über und akzeptiert somit die Eingabe.

Turingmaschinen: Eingeschränkte Varianten - EA

Endlicher Automat: Turingmaschine, die das Arbeitsband gar nicht benötigt. Übergänge besitzen einfache Form $(q, a; p, D_E)$

Erlaubt man nur $D_E \in \{R, S\}$, kann die Übergangsfunktion δ durch Tripel der Gestalt $(q, a; p)$ beschreiben werden ($a \in T \cup \{\varepsilon\}$); dabei ist

$(q, a; p)$ für $a \in T$ als $(q, a; p, R)$ zu interpretieren und
$(q, \varepsilon; p)$ als $(q, a; p, S)$ für ein beliebiges $a \in T$.

Außerdem geht man davon aus, dass ein endlicher Automat seine Analyse auf dem ersten Symbol des Eingabewortes beginnt und in einem Endzustand akzeptiert, wenn der Lesekopf auf dem rechten Begrenzungssymbol steht.

Akzeptierte Sprachen

Eine formale Sprache heißt

0. **rekursiv aufzählbar**[3], wenn Sie von einer *Turingmaschine* akzeptiert wird
1. **kontextsensitiv**, wenn sie von einem *linear beschränkten Automaten (LBA)* akzeptiert wird
2. **kontextfrei**, wenn sie von einem *Kellerautomaten* akzeptiert wird
3. **regulär**, wenn Sie von einem *endlichen Automaten* akzeptiert wird.

[3] engl: recursively enumerable, auch bezeichnet als: Turing-erkennbar (Turing-recognizable), semi-entscheidbar (semi-decidable)

Reguläre Sprachen

- Gebildet durch Vereinigung, Konkatenation und Stern
- Äquivalent: endliche Automaten, reguläre Grammatiken

Anwendungen in der Informatik:

- Compilerbau: *Tokens* bilden reguläre Sprache, verarbeitet durch Scanner (Lexer).
  Reguläre Ausdrücke dienen als Eingabe für Scannergeneratoren (`lex`, `flex`).
- Texteditoren: erweiterte Suche
- Unix-Shells, `grep`, `awk`, Perl, Xml, ...

Reguläre Sprachen

Reguläre Mengen (Sprachen)

Die Menge $\mathcal{L}_{\text{reg}}(\Sigma)$ der regulären Mengen (Sprachen) über Σ ist die kleinste Menge, sodass

- $\{\}, \{a\} \in \mathcal{L}_{\text{reg}}(\Sigma)$ für alle $a \in \Sigma$
- $A, B \in \mathcal{L}_{\text{reg}}(\Sigma) \Rightarrow A \cup B, A \cdot B, A^* \in \mathcal{L}_{\text{reg}}(\Sigma)$

Reguläre Ausdrücke (Algebraische Notation)

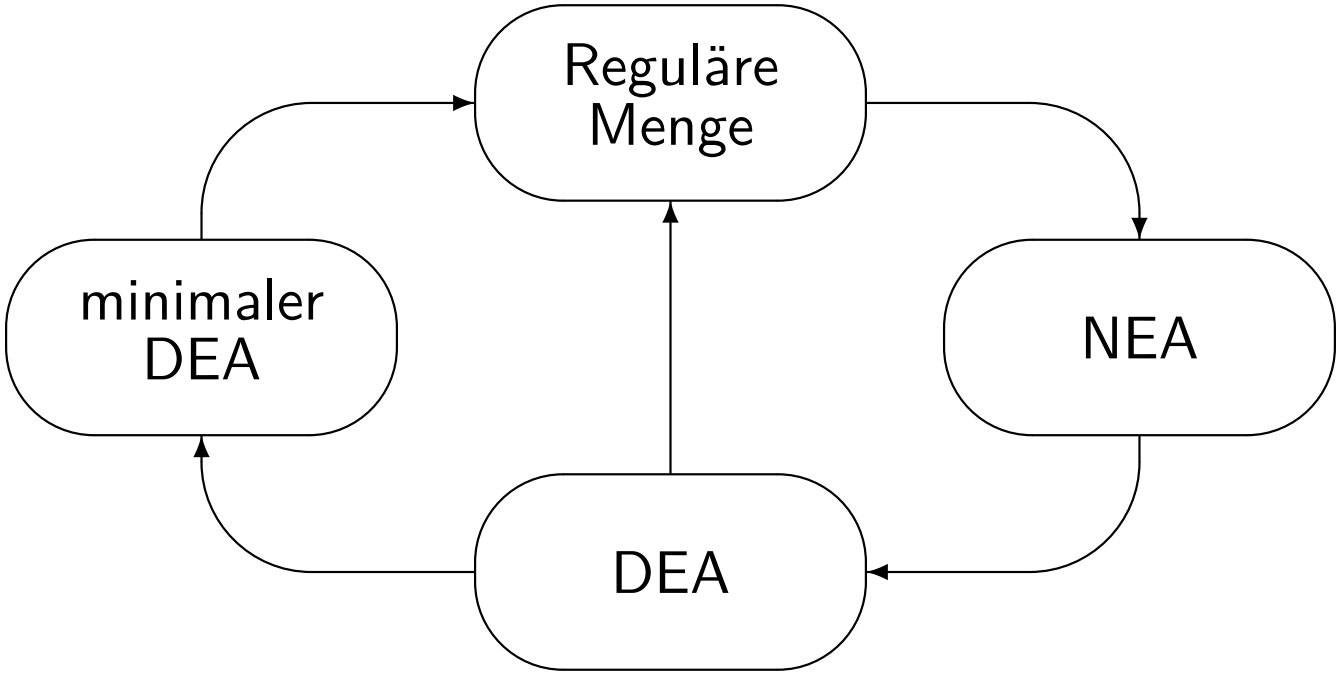
s	statt	$\{s\}$ für $s \in \Sigma$	$L_1 + L_2$	statt	$L_1 \cup L_2$
ε	statt	$\{\varepsilon\}$	$L_1 L_2$	statt	$L_1 \cdot L_2$
$\emptyset$	statt	$\{\}$	L^*	bleibt	L^*

* hat die höchste Priorität, $+$ die niedrigste.

Anmerkung: Genau genommen, ist ein regulärer Ausdruck E keine Sprache. Will man auf die Sprache Bezug nehmen, die von E beschrieben wird, sollte man eigentlich $L(E)$ verwenden.

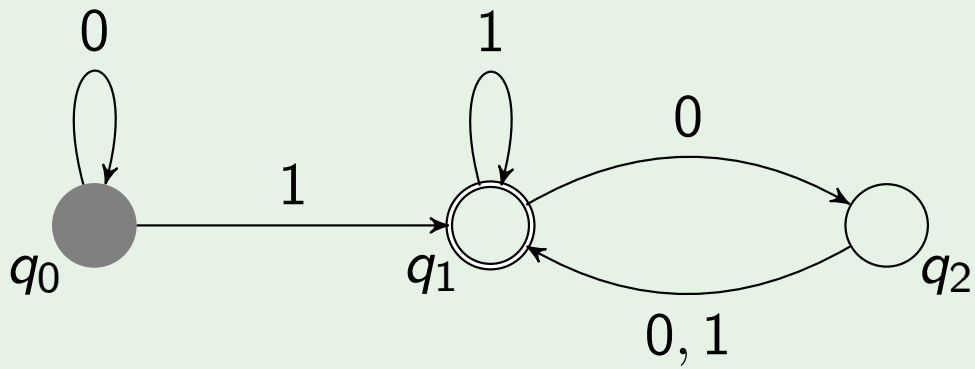
Endliche Automaten

Modell für Systeme mit Ein/Ausgaben aus endlichem Wertebereich und mit endlichem Speicher.

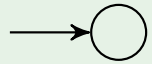


DEA ... deterministischer endlicher Automat
NEA ... nichtdeterministischer endlicher Automat

DEA: Beispiel



Zustände: q_0, q_1, q_2

Startzustand:  (oder auch )

Endzustand: 

Transitionen (Übergänge): Pfeile

Eingabe: Wort; Ausgehend vom Startzustand liest der Automat $\mathcal{A}$ von links nach rechts Symbol für Symbol. Nach dem Lesen eines Symbols geht $\mathcal{A}$ in den nächsten Zustand über indem er entlang der mit diesem Symbol markierten Kante geht. Nach dem Lesen des letzten Symbols wird der "Output" erzeugt: Befindet sich $\mathcal{A}$ in einem Endzustand, wird das Wort akzeptiert; ansonsten wird das Wort nicht akzeptiert.

Deterministische endliche Automaten (DEA)

DEA

Ein deterministischer endlicher Automat (DEA) ist ein 5-Tupel

$$\mathcal{A} = (Q, \Sigma, \delta, q_0, F), \text{wobei}$$

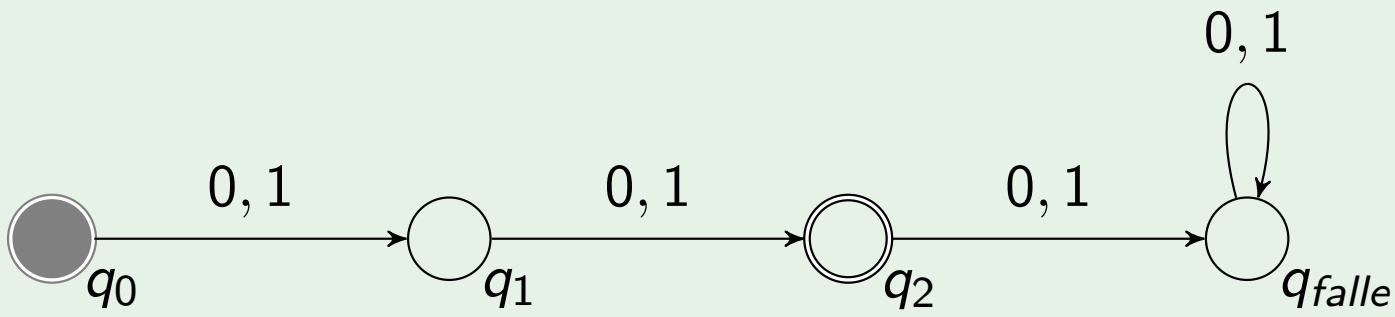
Q ... endliche Menge von Zuständen
Σ ... Eingabealphabet
$\delta\colon Q \times \Sigma \to Q$ ... Übergangsfunktion (total)
$q_0 \in Q$ ... Anfangszustand
$F \subseteq Q$ ... Menge von Endzuständen

DEA: Falle

Falle

Sei $\mathcal{A} = (Q, \Sigma, \delta, q_0, F)$ ein DEA und $q \in Q - F$ mit $\delta(q, a) = q$ für alle $a \in \Sigma$. Dann heißt q *Falle*.

Beispiel: Falle



Akzeptierte Sprache: $\{\varepsilon, 00, 01, 10, 11\}$

Deterministischer endlicher Automat

Um das Verhalten eines DEA auf einer Zeichenkette formal zu beschreiben, erweitern wir die Übergangsfunktion δ auf beliebige Wörter aus Σ^*:

Erweiterte Übergangsfunktion

$\delta^* : Q \times \Sigma^* \to Q$

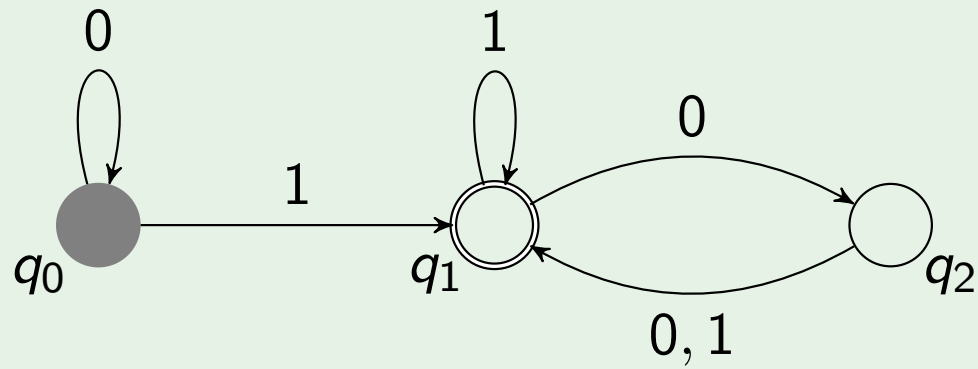
$$\delta^*(q, \varepsilon) = q, \qquad \delta^*(q, aw) = \delta^*(\delta(q, a), w)$$

für alle $q \in Q$, $w \in \Sigma^*$, $a \in \Sigma$.

Akzeptierte Sprache

$\mathcal{L}(\mathcal{A}) = \{w \in \Sigma^* \mid \delta^*(q_0, w) \in F\}$

DEA: Beispiel



$$\mathcal{A} = (Q, \Sigma, \delta, q_0, F), \text{wobei}$$

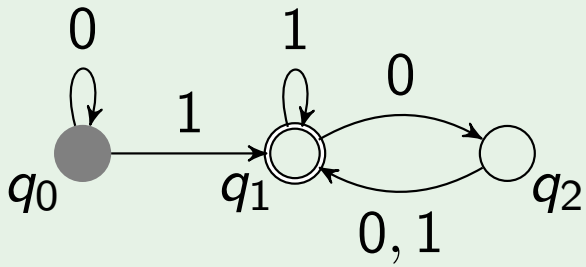
$Q = \{q_0, q_1, q_2\}$
$\Sigma = \{0, 1\}$
δ geg. durch Übergangsmatrix:
q_0 *Startzustand*
$F = \{q_1\}$

δ	0	1
q_0	q_0	q_1
q_1	q_2	q_1
q_2	q_1	q_1

$$\mathcal{L}(\mathcal{A}) = \{0\}^*\{1\}(\{1\}^*\{00, 01\})^*\{1\}^*$$

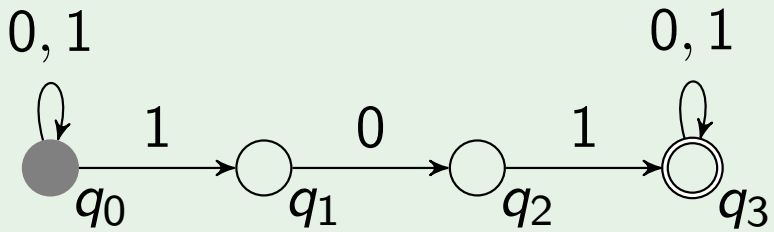
Nichtdeterministische endliche Automaten (NEA)

DEA



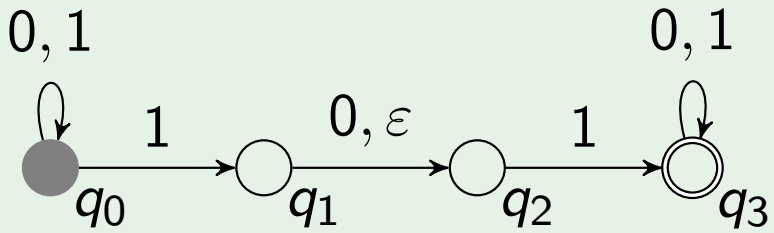
Von einem Zustand aus gibt es mit ein und demselben Eingabesymbol genau einen Folgezustand.

NEA



Von einem Zustand aus kann es mit ein und demselben Eingabesymbol mehrere Folgezustände geben.

ε-NEA



Von einem Zustand aus kann es mit ein und demselben Eingabesymbol mehrere Folgezustände geben, Übergänge sind auch mit ε möglich (ε-Übergänge).

Nichtdeterministische endliche Automaten (NEA)

Übergangsfunktion NEA: $\delta\colon Q \times \Sigma \to \mathcal{P}(Q)$

Übergangsfunktion ε-NEA: $\delta\colon Q \times (\Sigma \cup \{\varepsilon\}) \to \mathcal{P}(Q)$

Erweiterte Übergangsfunktion: $\delta^*\colon Q \times \Sigma^* \to \mathcal{P}(Q)$

$$\delta^*(q, w) = \{q' \in Q \mid q \rightsquigarrow wq'\}$$

$q \rightsquigarrow wq'$ ... es gibt einen mit w beschrifteten Pfad von q nach q'

Akzeptierte Sprache:

$$\mathcal{L}(\mathcal{A}) = \{w \in \Sigma^* \mid \delta^*(q_0, w) \cap F \neq \{\}\}$$

Automaten $\mathcal{A}$ und $\mathcal{A}'$ sind äquivalent, falls $\mathcal{L}(\mathcal{A}) = \mathcal{L}(\mathcal{A}')$.

Determinisierung (NEA $\rightarrow$ DEA)

Zu jedem NEA gibt es einen äquivalenten DEA.

Beweis (Teilmengenkonstruktion)

NEA: $\mathcal{A} = (Q, \Sigma, \delta, q_0, F)$
DEA: $\hat{\mathcal{A}} = (\hat{Q}, \Sigma, \hat{\delta}, \hat{q}_0, \hat{F})$, wobei

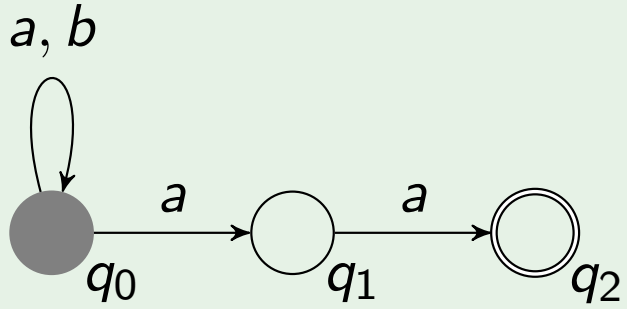
$$
\begin{aligned}
\hat{Q} &= \mathcal{P}(Q) \\
\hat{\delta}(\hat{q}, a) &= \bigcup_{q \in \hat{q}} \delta^*(q, a) \qquad \text{für alle } \hat{q} \in \hat{Q},\ a \in \Sigma \\
\hat{q}_0 &= \{q_0\} \\
\hat{F} &= \begin{cases} \{\hat{q} \in \hat{Q} \mid \hat{q} \cap F \neq \{\}\} \cup \{\hat{q}_0\} & \textit{falls } \varepsilon \in \mathcal{L}(\mathcal{A}) \\ \{\hat{q} \in \hat{Q} \mid \hat{q} \cap F \neq \{\}\} & \textit{sonst} \end{cases}
\end{aligned}
$$

□

Tipp: Berechne die Übergangsfunktion ausgehend von $\hat{q}_0$ nur für tatsächlich erreichbare Zustände.

Determinisierung: Beispiel

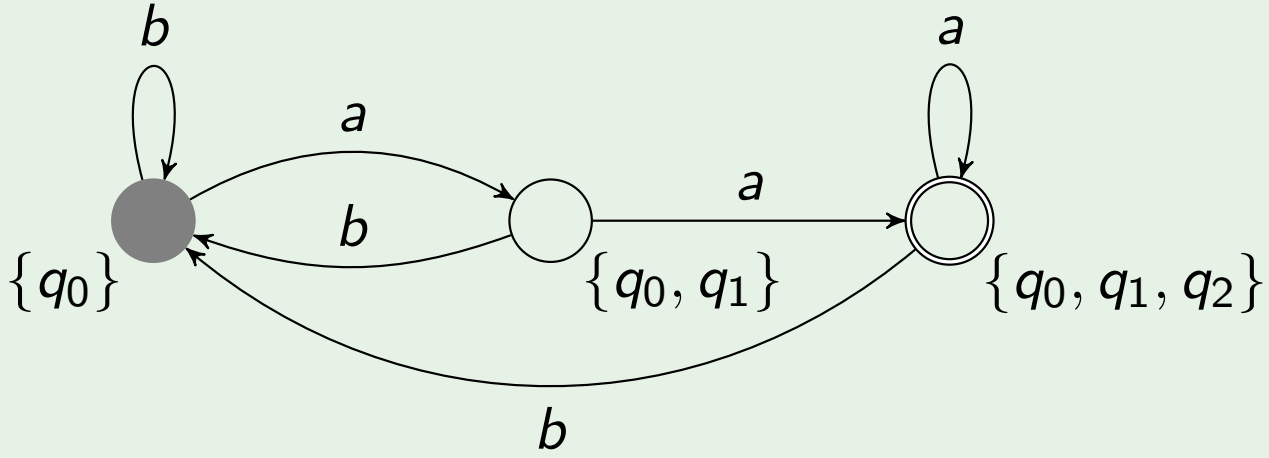
NEA:



δ	a	b
q_0	$\{q_0, q_1\}$	$\{q_0\}$
q_1	$\{q_2\}$	$\{\}$
q_2	$\{\}$	$\{\}$

	$\hat{\delta}$	a	b
SZ	$\{q_0\}$	$\{q_0, q_1\}$	$\{q_0\}$
	$\{q_0, q_1\}$	$\{q_0, q_1, q_2\}$	$\{q_0\}$
EZ	$\{q_0, q_1, q_2\}$	$\{q_0, q_1, q_2\}$	$\{q_0\}$

DEA:



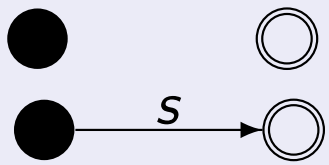
Reguläre Menge $\rightarrow$ ε-NEA

Zu jeder regulären Sprache L gibt es einen endlichen Automaten $\mathcal{A}$, sodass $L = \mathcal{L}(\mathcal{A})$.

Beweis

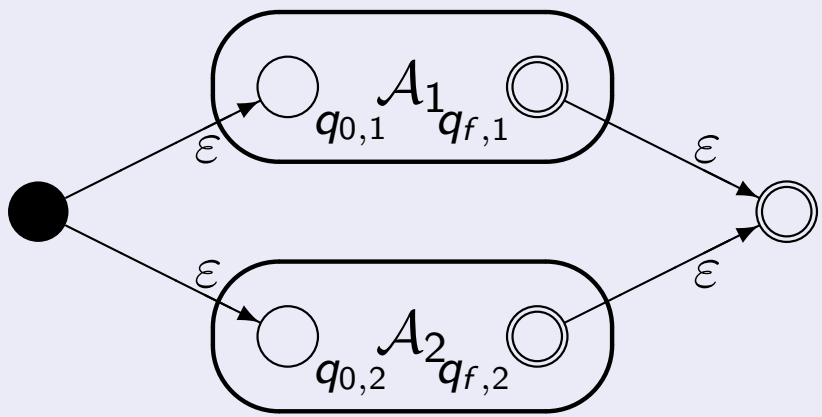
$L = \{\}$:

$L = \{s\}$:



$s \in (\Sigma \cup \{\varepsilon\})$

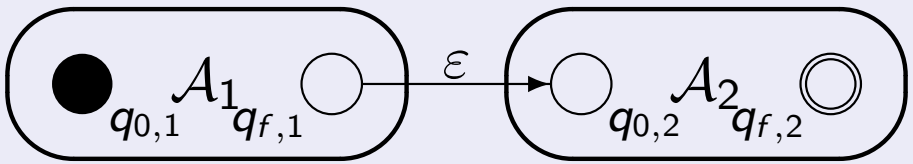
$L = L_1 \cup L_2$:



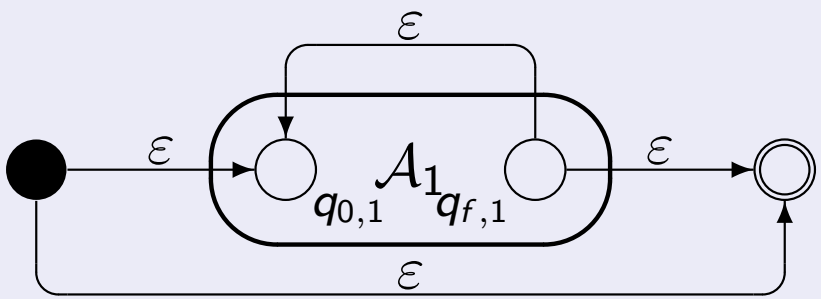
Reguläre Menge $\rightarrow$ ε-NEA

Beweis ctd.

$L = L_1 \cdot L_2$:



$L = (L_1)^*$:



□

DEA $\rightarrow$ reguläre Menge

Jede von einem DEA akzeptierte Sprache ist regulär.

Beweis

$\mathcal{A} = (\{q_1, \ldots, q_n\}, \Sigma, \delta, q_1, F)$
R_{ij}^k ... Menge aller Wörter, mit denen man von q_i nach q_j gelangt, ohne einen Zustand mit einem Index größer als k zu berühren.

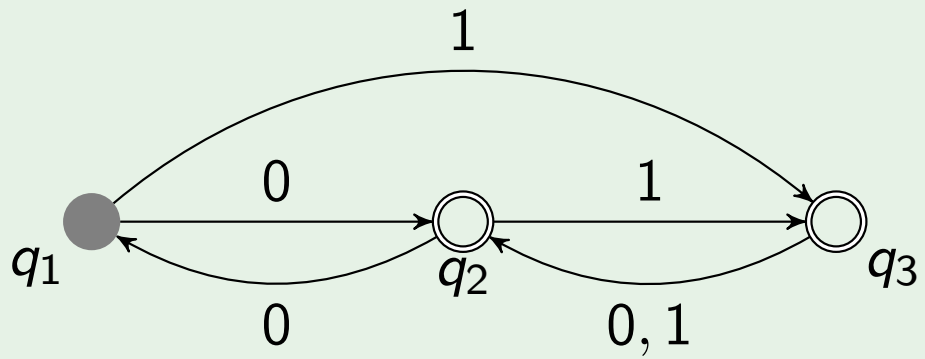
$$R_{ij}^0 = \begin{cases} \{s \in \Sigma \mid \delta(q_i, s) = q_j\} & i \neq j \\ \{s \in \Sigma \mid \delta(q_i, s) = q_j\} \cup \{\varepsilon\} & i = j \end{cases}$$

$$R_{ij}^k = R_{ij}^{k-1} \cup R_{ik}^{k-1} \cdot (R_{kk}^{k-1})^* \cdot R_{kj}^{k-1} \quad k > 0$$

$$\mathcal{L}(\mathcal{A}) = \bigcup_{q_j \in F} R_{1j}^n$$

□

Von DEA zu Regulären Mengen: Beispiel



	$k = 0$	$k = 1$	$k = 2$
R_{11}^k	$\{\varepsilon\}$	$\{\varepsilon\}$	$\{00\}^*$
R_{12}^k	$\{0\}$	$\{0\}$	$\{0\} \cdot \{00\}^*$
R_{13}^k	$\{1\}$	$\{1\}$	$\{0\}^* \cdot \{1\}$
R_{21}^k	$\{0\}$	$\{0\}$	$\{0\} \cdot \{00\}^*$
R_{22}^k	$\{\varepsilon\}$	$\{\varepsilon, 00\}$	$\{00\}^*$
R_{23}^k	$\{1\}$	$\{1, 01\}$	$\{0\}^* \cdot \{1\}$
R_{31}^k	$\{\}$	$\{\}$	$\{0, 1\} \cdot \{00\}^* \cdot \{0\}$
R_{32}^k	$\{0, 1\}$	$\{0, 1\}$	$\{0, 1\} \cdot \{00\}^*$
R_{33}^k	$\{\varepsilon\}$	$\{\varepsilon\}$	$\{\varepsilon\} \cup \{0, 1\} \cdot \{0\}^* \cdot \{1\}$

Minimalautomat

Zu jeder regulären Sprache kann man effektiv einen DEA mit einer minimalen Anzahl von Zuständen konstruieren, der bis auf die Umbenennung der Zustände eindeutig ist.

Minimierungsalgorithmus von Brzozowski[4]

Sei $A = (Q, \Sigma, \delta, q_0, F)$ ein DEA mit $L(A) = L$. Dann ist $A^r = (Q, \Sigma, \{(q, a, p) \mid (p, a, q) \in \delta\}, F, \{q_0\})$ mit $L(A^r) = L^r$.

Minimierungsalgorithmus von Brzozowski

Gegeben ein DEA A. Um den zu A äquivalenten Minimalautomaten C zu erhalten:

1. konstruiere einen NEA A^r durch "Spiegelung" von A
2. konstruiere einen DEA B durch Determinisierung von A^r
3. konstruiere einen NEA B^r durch "Spiegelung" von B
4. konstruiere einen DEA C durch Determinisierung von B^r

[4] Janusz (John) Antoni Brzozowski (1935-2019)

Abschlusseigenschaften regulärer Sprachen

$\mathcal{L}_{\text{reg}}(\Sigma)$ ist **abgeschlossen** gegenüber:

- Vereinigung, Verkettung, Stern-Operator: per Definition.
- Plus-Operator: $A^+ = A^* \cdot A$.
- Komplement bzgl. Σ^*: konstruiere DEA und vertausche End- und Nichtendzustände. (Falle nicht vergessen!)
- Durchschnitt: $A \cap B = \overline{\overline{A} \cup \overline{B}}$.
- Differenz: $A - B = A \cap \overline{B}$.
- Spiegelung: Konstruiere EA, vertausche Start- mit Endzustand, kehre alle Zustandsübergänge um.
- Homomorphismen: repräsentiere L durch regulären Ausdruck und ersetze alle Vorkommnisse von a durch Ausdruck für $h(a)$.

Homomorphismus

Seien Σ und Γ zwei Alphabete. **Homomorphismus**: $h : \Sigma \longrightarrow \Gamma^*$
Induktive Erweiterung auf Wörter über Σ:

1. $h(\varepsilon) = \varepsilon$
2. $h(wa) = h(w)h(a)$ für alle $w \in \Sigma^*$ und $a \in \Sigma$.

Anwendung von h auf jedes Wort der Sprache L:

$$h(L) = \{h(w) \mid w \in L\}$$

Beispiel

Sei $h : \{0,1\}^* \longrightarrow \{a,b\}^*$ ein Homomorphismus mit $h(0) = ab$ und $h(1) = \varepsilon$

Dann ist $h(0011) = abab$ und $h(L(10^*1)) = L((ab)^*)$

Entscheidungsprobleme

Folgende Probleme sind für reguläre Sprachen L, L' **entscheidbar**:

- Gehört ein Wort w der Sprache L an? (**Wortproblem**)
  Konstruiere einen DEA für L und prüfe, ob $\delta^*(q_0, w) \in F$.
- Ist L leer?
  Konstruiere einen DEA für L und prüfe, ob von q_0 aus ein Endzustand erreichbar ist.
- Ist L endlich oder unendlich?
  L ist unendlich gdw. der minimale DEA für L (abgesehen von der Falle!) einen Zyklus enthält.
- Gilt $L = L'$?
  Überprüfe, ob $L - L'$ und $L' - L$ leer sind.

Grenzen der Regularität

Sei $L = \{0^n 1^n \mid n \geq 0\}$. Gibt es einen DEA für L?

Es sieht so aus als müsste sich die Maschine die Anzahl der Nullen merken. Nachdem ebendiese Anzahl aber nicht limitiert ist, müsste sich der DEA eine unbeschränkte Anzahl von Möglichkeiten merken können. Mit einer endlichen Anzahl von Zuständen ist das aber nicht möglich!

Aber: Nur weil es so aussieht, als ob eine Sprache unbegrenzten Speicher brauchen würde, ist das nicht notwendigerweise so!

Regulär oder nicht?

$A = \{w \mid w \text{ enthält gleich viele Symbole 0 wie Symbole 1}\}$
$B = \{w \mid \text{die Teilwörter}^5 \text{ 01 und 10 kommen gleich oft in } w \text{ vor}\}$

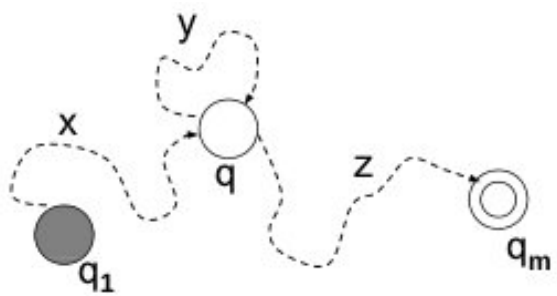
[5] Für ein Wort $w \in \Sigma^*$, wobei $w = xuy$ für Wörter $x, u, y \in \Sigma^*$ heißt u **Teilwort** von w.

Schubfachprinzip[6]

Schubfachprinzip (pigeonhole principle)

Bei Verteilung von $n+1$ Gegenständen auf n Schubfächer müssen in mindestens einem Schubfach zwei Gegenstände landen.

DEA $\mathcal{A}$ habe m Zustände. Gilt für ein von $\mathcal{A}$ akzeptiertes Wort w, dass $|w| \geq m$, so muss mindestens ein Zustand mehr als einmal durchlaufen werden. (Schubfachprinzip!)
Zerlege w in xyz: y gehe von q nach q; diese Schleife kann ausgelassen bzw. beliebig oft wiederholt werden, d.h. xy^iz wird für alle i ebenfalls von $\mathcal{A}$ akzeptiert!



[6] Johann Peter Gustav Lejeune Dirichlet, 1805-1859

Pumping Lemma für reguläre Sprachen

(Unendliche) Reguläre Sprachen haben eine spezielle Eigenschaft: Jedes Wort ab einer bestimmten Länge kann "aufgepumpt" werden, d.h. jedes solche Wort enthält ein Teilstück, das beliebig oft wiederholt werden kann, wobei die resultierenden Wörter noch immer in derselben Sprache liegen.

Pumping Lemma für reguläre Sprachen

Sei L eine unendliche reguläre Sprache. Dann **gibt es** eine (nur von L abhängige) Schranke $m > 0$ so, dass für **jedes** Wort $w \in L$ mit $|w| \geq m$ Wörter x, y, z so **existieren**, dass

$$w = xyz \qquad \text{mit } |xy| \leq m \text{ und } |y| > 0$$

sowie

$$w_i = xy^i z \in L \qquad \textbf{für alle } i \geq 0.$$

Reguläres Pumping Lemma - Beispiel

Zu zeigen: $L = \{a^n b^n \mid n \geq 0\}$ ist nicht regulär.

Beweis durch Widerspruch:

Angenommen L sei regulär. Für **beliebiges** m (Konstante aus dem Pumping Lemma) wähle **ein** $w \in L$ mit $|w| \geq m$, z.B.

$$w = a^m b^m$$

Betrachte **beliebige** Zerlegungen von w in xyz, wobei $|xy| \leq m$ und $|y| > 0$: xy kann nur aus Symbolen a bestehen.

Wähle **ein** i so, dass $xy^i z$ nicht von der Gestalt $a^n b^n$ ist:

z.B. $i = 2$, dann müsste – wäre L regulär – auch $xy^2 z = a^{m+|y|} b^m$ aus L sein, was aber nicht der Fall ist! Widerspruch! L kann nicht regulär sein.

Grammatiken: Formale Definition

Grammatik

Grammatik $G = (N, T, P, S)$, wobei

N ... endliche Menge von Nonterminalen (Variablen)
T ... endliche Menge von Terminalsymbolen ($N \cap T = \{\}$)
$P \subseteq (N \cup T)^+ \times (N \cup T)^*$ ... Produktionen
$S \in N$ ... Startsymbol

Wir schreiben
$\alpha \to \beta$ statt $(\alpha, \beta) \in P$
$\alpha \to \beta_1 \mid \cdots \mid \beta_n$ statt $\alpha \to \beta_1, \ldots, \alpha \to \beta_n$

Ableitungen

Direkte Ableitung:

$x\alpha y \Rightarrow x\beta y$ falls $\alpha \to \beta \in P$ und $x, y \in (N \cup T)^*$

Ableitung von w aus v **in** n **Schritten**:

Es gibt Wörter $w_0, w_1, \ldots, w_n$, sodass $v = w_0$, $w = w_n$ und $w_{i-1} \Rightarrow w_i$ für $1 \leq i \leq n$.
Schreibweise: $v \Rightarrow w_1 \Rightarrow \cdots \Rightarrow w_{n-1} \Rightarrow w$ bzw. $v \Rightarrow^n w$

Ableitung in mehreren Schritten:

$v \Rightarrow^* w$ ... reflexiver und transitiver Abschluss von $\Rightarrow$

Satzformen und erzeugte Sprache

Gilt $S \Rightarrow w$ für ein Wort $w \in (N \cup T)^*$, so nennt man w **Satzform**.

Menge aller in n Schritten ableitbaren Satzformen:
$SF(G, n) = \{w \in (N \cup T)^* | S \Rightarrow^n w\}$

Erzeugte Sprache

Von G erzeugte *Sprache* $\mathcal{L}(G)$:

$$\mathcal{L}(G) \quad = \quad \{w \in T^* \mid S \Rightarrow^* w\}$$

Grammatiken G_1 und G_2 heißen äquivalent, wenn $\mathcal{L}(G_1) = \mathcal{L}(G_2)$ gilt.

Grammatiken: Beispiele

$\{a^n \mid n \geq 0\}$

$$G_1 = (\{S\}, \{a\}, \{S \to \varepsilon, S \to aS\}, S)$$

$$\mathcal{L}(G_1) = \{\varepsilon\} \cup \{a^n \mid n \geq 1\} = \{a\}^*$$

Alle in G_1 möglichen Ableitungen sind von der Gestalt

$$S \Rightarrow \varepsilon \quad \text{bzw.} \quad S \Rightarrow aS \Rightarrow^n a^{n+1}S \Rightarrow a^{n+1}$$

für ein $n \geq 0$.

Grammatiken: Beispiele

$\{a^n b^n | n \geq 0\}$

$$G_2 = (\{S\}, \{a, b\}, \{S \to aSb, S \to \varepsilon\}, S)$$

$$\mathcal{L}(G_2) = \{a^n b^n | n \geq 0\}$$

Alle in G_2 möglichen Ableitungen sind von der Gestalt

$$S \Rightarrow^n a^n S b^n \Rightarrow a^n b^n \text{ für alle } n \geq 0$$

Formaler Beweis mittels *Induktion*:
Menge aller Satzformen nach genau $n \geq 1$ Schritten: $SF(G_2, n) = \{a^n S b^n, a^{n-1} b^{n-1}\}$

Induktionsbasis:	$SF(G_2, 1) = \{a^1 S b^1, \varepsilon\}$
Induktionshypothese:	$SF(G_2, n) = \{a^n S b^n, a^{n-1} b^{n-1}\}$
Induktionsbehauptung:	$SF(G_2, n+1) = \{a^{n+1} S b^{n+1}, a^n b^n\}$

Beweis: Das Wort $a^{n-1} b^{n-1}$ ist terminal und daher nicht mehr weiter ableitbar. Aus $a^n S b^n$ ist ableitbar:

mittels $S \to aSb$: $a^{n+1} S b^{n+1}$ mittels $S \to \varepsilon$: $a^n b^n$

Grammatiken: Beispiele

$\{a^n b^n c^n | n \geq 1\}$

$$G_3 = (\{S, A, C\}, \{a, b, c\}, P_3, S) \text{wobei}$$

$$\begin{aligned} P_3 \quad = \quad & \{S \to abc, S \to aAbc, A \to aAbC, A \to abC, \\ & Cb \to bC, Cc \to cc\} \end{aligned}$$

$$\mathcal{L}(G_3) = \{a^n b^n c^n | n \geq 1\}$$

Alle in G_3 möglichen Ableitungen sind von der Gestalt $S \Rightarrow abc$ bzw. für $n \geq 2$:

$$S \Rightarrow aAbc \Rightarrow^{n-2} a^{n-1} A (bC)^{n-2} bc \Rightarrow a^n (bC)^{n-1} bc \Rightarrow^* a^n b^n c^n$$

Grammatik-Typen: Typ-i-Grammatiken

Typ-i-Grammatiken

Sei $G = (N; T; P; S)$ eine Grammatik. Dann heißt G auch **unbeschränkte Grammatik** (**Typ-0**).
Gilt für alle Produktionen $\alpha \to \beta \in P$

- $|\alpha| \leq |\beta|$ so heißt G **monoton**;
- $\alpha = uAv$ und $\beta = uwv$ für ein $A \in N$, $w \in (N \cup T)^+$ und $u, v \in (N \cup T)^*$ so heißt G **kontextsensitiv** (**Typ-1**);
- $A \to \beta$ für ein $A \in N$, so heißt G **kontextfrei** (**Typ-2**);
- $A \to aB$ oder $A \to \varepsilon$ für $A, B \in N$ und $a \in T$, so heißt G **regulär** (**Typ-3**).

Für monotone und kontextsensitive Grammatiken gilt: Kommt S nicht auf der rechten Seite einer Produktion vor, so ist auch die Produktion $S \to \varepsilon$ erlaubt.

Erzeugte Sprachen

Erzeugte Sprachen

Eine formale Sprache heißt **rekursiv aufzählbar**, **monoton**, **kontextsensitv**, **kontextfrei** bzw. **regulär**, wenn sie von einer Typ-0-, monotonen, Typ-1-, Typ-2-, bzw. Typ-3-Grammatik erzeugt wird.

Aufgrund der Definition können wir nun die einzelnen Sprachen aus den vorigen Beispielen klassifizieren:

Es ergibt sich, dass
$\mathcal{L}(G_1) = \{a^n | n \geq 0\}$ regulär
$\mathcal{L}(G_2) = \{a^n b^n | n \geq 0\}$ kontextfrei und
$\mathcal{L}(G_3) = \{a^n b^n c^n | n \geq 1\}$ monoton ist.

Reguläre Grammatiken

Reguläre Grammatiken

Eine Grammatik heißt *regulär*, wenn alle Produktionen von der Form

$$A \to aB \qquad \text{oder} \qquad A \to \varepsilon$$

sind ($A, B \in N$, $a \in T$).

$L = \{a\}^*$

$$G = (\{S\}, \{a\}, \{S \to aS \mid \varepsilon\}, S)$$

$L = \{a\}^+$

$$G = (\{S, T\}, \{a\}, \{S \to aT, T \to aT \mid \varepsilon\}, S)$$

Reguläre Grammatiken und Sprachen

Eine Sprache ist genau dann regulär, wenn sie von einer regulären Grammatik erzeugt wird. Umgekehrt lässt sich zu jeder regulären Sprache eine reguläre Grammatik finden, die sie generiert.

Beweis:
Simuliere DEA mittels regulärer Grammatik und umgekehrt.

DEA und reguläre Grammatik

DEA: $\mathcal{A} = (Q, \Sigma, \delta, q_0, F)$

definiere reguläre Grammatik

$$G = (Q, \Sigma, P, q_0)$$

wobei P wie folgt definiert wird:

$$\begin{array}{ll} q \to ap \in P & \text{wenn } \delta(q, a) = p \\ q \to \varepsilon \in P & \text{wenn } q \in F \end{array}$$

Es folgt dann: $\delta^*(q_0, w) \in F$ g.d.w. $q_0 \Rightarrow^* w$

□

DEA und reguläre Grammatik: Beispiel

$\mathcal{A} = (\{q_0, q_1\}, \{0, 1\}, \delta, q_0, \{q_1\})$ mit

$$\begin{aligned} \delta(q_0, 0) = q_0,\ \delta(q_0, 1) = q_1, \\ \delta(q_1, 0) = q_1,\ \delta(q_1, 1) = q_0 \end{aligned}$$

Definiere $G = (\{q_0, q_1\}, \{0, 1\}, P, q_0)$ mit

$$\begin{aligned} P \quad = \quad & \{q_0 \to 0\ q_0 \mid 1\ q_1, \\ & q_1 \to 0\ q_1 \mid 1\ q_0 \mid \varepsilon\} \end{aligned}$$

Ableitung von 0 1 0:

$$q_0 \Rightarrow 0\ q_0 \Rightarrow 0\ 1\ q_1 \Rightarrow 0\ 1\ 0\ q_1 \Rightarrow 0\ 1\ 0$$

Kontextfreie Grammatiken

Kontextfreie Grammatik

Eine Grammatik heißt *kontextfrei*, wenn alle Produktionen von der Form $A \to \beta$ sind, wobei $A \in N$ und $\beta \in (N \cup T)^*$.
Eine Sprache L heißt kontextfrei, wenn es eine kontextfreie Grammatik G gibt, sodass $L = \mathcal{L}(G)$.

Linksableitung:

$xAy \Rightarrow_L x\beta y$ falls $A \to \beta \in P$ und $x \in T^*$

Rechtsableitung:

$xAy \Rightarrow_R x\beta y$ falls $A \to \beta \in P$ und $y \in T^*$

Parallelableitung:

$x_0 A_1 x_1 \cdots A_n x_n \Rightarrow_P x_0 \beta_1 x_1 \cdots \beta_n x_n$ falls
$A_1 \to \beta_1, \ldots, A_n \to \beta_n \in P$ und $x_0, \ldots, x_n \in T^*$

Ableitungsvarianten

Alle Ableitungsvarianten ergeben dieselbe Sprache:

Ist G eine kontextfreie Grammatik, dann gilt:

$$\begin{aligned}
\mathcal{L}(G) &= \{w \in T^* \mid S \Rightarrow^* w\} \\
&= \{w \in T^* \mid S \Rightarrow_L^* w\} \\
&= \{w \in T^* \mid S \Rightarrow_R^* w\} \\
&= \{w \in T^* \mid S \Rightarrow_P^* w\}
\end{aligned}$$

Kontextfreie Grammatiken: Beispiele

Wohlgeformte Klammerausdrücke

WKA ist die kleinste Menge, sodass

- $\varepsilon \in WKA$
- $w \in WKA \Rightarrow (w) \in WKA$
- $w_1, w_2 \in WKA \Rightarrow w_1 w_2 \in WKA$

$$G = (\{A\}, \{(,)\}, \{A \to \varepsilon \mid (A) \mid A\,A\}, A)$$

Kontextfreie Grammatiken: Beispiele

Palindrome über $\{a, b\}$

$G = (\{S\}, \{a, b\},$
$\quad \{S \to \varepsilon \mid a \mid b \mid a\,S\,a \mid b\,S\,b\},\ S)$

$\mathcal{L}(G) = \{\varepsilon, a, b, aa, bb, aaa, aba, bab, bbb, \ldots\}$

$L = \{w \in \{a, b\}^* \mid |w|_a = 2 \cdot |w|_b\}$

$G = (\{S\}, \{a, b\}, P, S)$, wobei
$P = \{S \to a\,S\,a\,S\,b\,S \mid a\,S\,b\,S\,a\,S \mid b\,S\,a\,S\,a\,S \mid \varepsilon\}$

Kontextfreie Grammatiken: Beispiele

$L = \{a^m + b^n = c^{m+n} \mid m, n \geq 0\}$

Kontextfreie Grammatik $G = (\{S, T\}, \{a, b, c, +, =\}, P, S)$
wobei P:

$$P = \{ \begin{array}{lcl} S & \rightarrow & a\,S\,c \mid + \\ T & \rightarrow & b\,T\,c \mid = \end{array} \}$$

$aa + bb = cccc \in \mathcal{L}(G)$:

$$\begin{array}{lllll} S & \Rightarrow & a\,S\,c & \Rightarrow & aa\,S\,cc \\ & \Rightarrow & aa + b\,T\,ccc & \Rightarrow & aa + bb\,T\,cccc \end{array} \quad \begin{array}{ll} \Rightarrow & aa + T\,cc \\ \Rightarrow & aa + bb = cccc \end{array}$$

Einschub: Bäume

Baum

Ein (geordneter, markierter gerichteter) *Baum* über U und W ist ein Graph $g = (K, E, L)$ über U und W, der folgende Bedingungen erfüllt:

1. $W = \{1, ..., n\}$ für ein $n \geq 1$.
2. Es gibt genau einen ausgezeichneten Knoten p_0 (Wurzel), der keinen Vorgänger hat. Außerdem gibt es von der Wurzel aus zu jedem anderen Knoten q von g einen Pfad $(p_0, e_1, p_1, \ldots, p_{k-1}, e_k, q)$ der Länge $k \geq 1$, $(p_i, e_{i+1}, p_{i+1}) \in E$, $0 \leq i < k$, $q = p_k$.
3. Jeder von der Wurzel verschiedene Knoten hat genau einen Vorgänger.
4. Jeder Knoten ohne Nachfolger heißt Blatt.
5. Ist p kein Blatt, so sind die Nachfolger von p geordnet (die Kanten tragen die Bezeichnungen 1 bis k für ein k).

Ableitungsbäume für kontextfreie Grammatiken

Ableitungsbaum

Sei $G = (N, T, P, S)$ eine kontextfreie Grammatik. Ein Baum $g = (K, E, L)$ über $U = N \cup T \cup \{\varepsilon\}$ und $W = \{1, ..., n\}$ heißt *Ableitungsbaum* für G, wenn Folgendes gilt:

1. Ist p_0 die Wurzel von g, so gilt $L(p_0) = S$.
2. Ist p kein Blatt, so muss $L(p) \in N$ gelten.
3. Ist p ein Blatt mit $L(p) = \varepsilon$, so ist p der einzige Nachfolger seines Vorgängers.
4. Ist $\{(p, i, q_i) \mid i \in 1, ..., n\}$ die geordnete Menge der von p mit $L(p) = A$ wegführenden Kanten, so ist $A \to L(q_1) \dots L(q_k)$ eine Produktion in P.

A-Baum für G: für Wurzel gilt $L(p_0) = A$
Ein Ableitungsbaum ist ein S-Baum für G.

Front eines Ableitungsbaumes

Sei $G = (N, T, P, S)$ eine kontextfreie Grammatik und $g = (K, E, L)$ ein A-Baum für G.

Wir definieren nun eine Ordnungsrelation auf den Pfaden von g: Seien $P(j) = (p_{j,0}, e_{j,1}, p_{j,1}, \ldots, e_{j,k_j}, p_{j,k_j})$ für $j \in \{1, 2\}$ zwei voneinander verschiedene Pfade in g, die in der Wurzel beginnen (i.e., $p_{1,0} = p_{2,0} = p_0$) und zu einem Blatt von g führen, dann definieren wir $P_1 < P_2$ genau dann, wenn es ein $m \geq 1$ so gibt, dass $e_{1,i} = e_{2,i}$ für alle $1 \leq i < m$ und $e_{1,m} < e_{2,m}$.

Betrachten wir nun alle derartigen Pfade in g, so sind diese wohlgeordnet und sind $p_1, \ldots, p_k$ die Blätter dieser Pfade, so ist die **Front** von g durch $L(p_1) \ldots L(p_k)$ definiert.

Ableitungen und Ableitungsbäume

Sei $G = (N, T, P, S)$ eine kontextfreie Grammatik und $g = (K, E, L)$ ein A-Baum für G sowie $w \in (N \cup T)^*$.

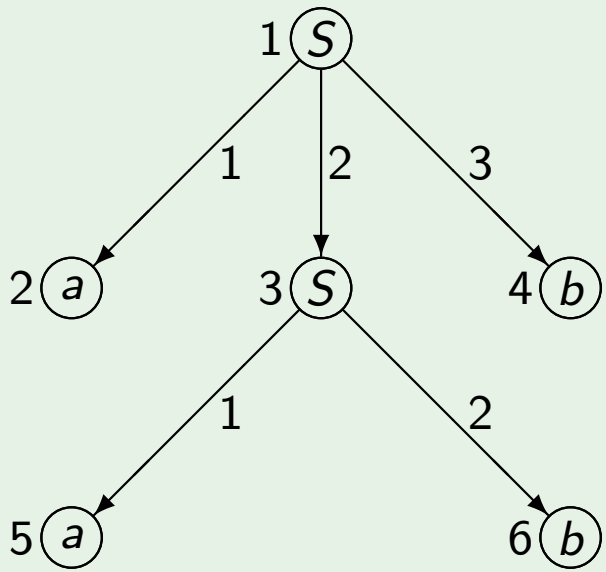
Dann gilt $A \Rightarrow^* w$ genau dann, wenn es einen A-Baum für G mit Front w gibt. Somit gilt für alle $w \in T^*$ auch $S \Rightarrow^* w$ genau dann, wenn es einen Ableitungsbaum für G mit Front w gibt.

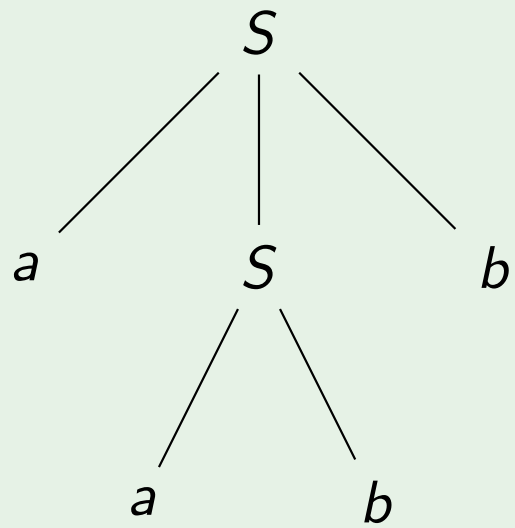
Jeder Linksableitung in G kann man eindeutig einen Ableitungsbaum zuordnen. Gibt es zwei verschiedene Linksableitungen in G für ein Wort w, so sind die entsprechenden Ableitungsbäume *nicht* äquivalent (im Sinne der Äquivalenz von Graphen).

Ableitungsbäume: Beispiel

$G = (\{S\}, \{a, b\}, \{S \to aSb, S \to ab\}, S)$

Ableitungsbaum für *aabb*:





Front des Ableitungsbaums:
ergibt sich als Folge der Labels der Knoten 2, 5, 6, 4 zu *aabb*

TIME FLIES LIKE AN ARROW
FRUIT FLIES LIKE A BANANA

Eindeutigkeit, (inhärente) Mehrdeutigkeit

Sei $G = (N, T, P, S)$ eine kontextfreie Grammatik. G heißt **eindeutig**, wenn es zu jedem in G ableitbaren Terminalwort genau eine Linksableitung in G gibt. Ansonsten heißt G **mehrdeutig**.

Eine kontextfreie *Sprache* L heißt **inhärent mehrdeutig**, wenn jede Grammatik, die L erzeugt, mehrdeutig ist.

Mehrdeutigkeit: Beispiel

If-then-else Anweisung

$G_1 = (\{Anw\}, \{if, then, else, expr, others\}, P_1, Anw)$

wobei P_1 folgende Produktionen enthält:

$$\begin{array}{rcl} Anw & \rightarrow & if\ expr\ then\ Anw \\ & | & if\ expr\ then\ Anw\ else\ Anw \\ & | & others \end{array}$$

G_1 ist mehrdeutig, da das Wort

$$if\ expr\ then\ if\ expr\ then\ others\ else\ others$$

zwei Linksableitungen besitzt.

Mehrdeutigkeit: Beispiel ctd.

$\mathcal{L}(G_1)$ ist aber nicht mehrdeutig, da es z.B. folgende eindeutige Grammatik mit $\mathcal{L}(G_1) = \mathcal{L}(G_2)$ gibt:

$G_2 = (\{Anw, AnwT, AnwTE\},$
$\{if, then, else, expr, others\}, P_2, Anw)$

wobei P_2 folgende Produktionen enthält:

$$
\begin{array}{lll}
Anw & \rightarrow & AnwT \\
 & | & AnwTE \\
AnwT & \rightarrow & if\ expr\ then\ Anw \\
 & | & if\ expr\ then\ AnwTE\ else\ AnwT \\
AnwTE & \rightarrow & if\ expr\ then\ AnwTE\ else\ AnwTE \\
 & | & others
\end{array}
$$

Inhärent mehrdeutige Sprachen: Beispiel

Die kontextfreie Sprache $L = L_1 \cup L_2$ mit

$L_1 = \{a^m b^m c^n \mid m, n \geq 1\}$ und $L_2 = \{a^m b^n c^n \mid m, n \geq 1\}$

ist inhärent mehrdeutig.
L ist kontextfrei:

$$\begin{array}{lcl} S & \to & S_1 \mid S_2 \\ S_1 & \to & S_1\, c \mid A \\ A & \to & a\, A\, b \mid \varepsilon \end{array} \qquad \begin{array}{lcl} S_2 & \to & a\, S_2 \mid B \\ B & \to & b\, B\, c \mid \varepsilon \end{array}$$

Grammatik ist mehrdeutig, da $a^n b^n c^n$ zwei verschiedene Ableitungen besitzt.
Man kann zeigen, dass *alle* Grammatiken für L mehrdeutig sind, d.h., L ist inhärent mehrdeutig.

Bemerkung: $L_1 \cap L_2 = \{a^n b^n c^n \mid n \geq 1\}$

Transformation von Produktionen

Nutzlose Produktionen: Entfernen aller Produktionen, die vom Startsymbol aus nicht erreichbar sind oder aus denen kein $w \in T^*$ ableitbar ist.

ε-Produktionen: Entfernen aller Produktionen $A \to \varepsilon$

Einheitsproduktionen: Entfernen aller Produktionen $A \to B$

Zu jeder kontextfreien Sprache ohne Leerwort gibt es eine kontextfreie Grammatik ohne nutzlose, ohne ε- und ohne Einheitsproduktionen. Eine solche Grammatik nennt man auch **reduzierte Grammatik**.

Chomsky[7] Normalform

Chomsky Normalform

Alle Produktionen besitzen die Form

$$A \to B\,C \qquad \text{oder} \qquad A \to a \qquad (a \in T,\ A, B, C \in N)\ .$$

Um das Leerwort erzeugen zu können, ist $S \to \varepsilon$ erlaubt, sofern S nicht auf der rechten Seite einer Produktion vorkommt.

Zu jeder kontextfreien Grammatik G kann man effektiv eine äquivalente Grammatik G' konstruieren, sodass $G' = \langle N', T', P', S' \rangle$ in Chomsky NF ist.

[7] Noam Chomsky, *1928

Greibach[8] Normalform für kf Grammatiken

Greibach Normalform:

Alle Produktionen haben die Form

$$A \to a\,w \qquad (a \in T,\ A \in N,\ w \in N^*)\ .$$

Erweiterte Greibach Normalform:

Alle Produktionen haben die Form

$$A \to a\,w \qquad (a \in T,\ A \in N,\ w \in (N \cup T)^*)\ .$$

Um das Leerwort erzeugen zu können, ist $S \to \varepsilon$ erlaubt, sofern S nicht auf der rechten Seite einer Produktion vorkommt.

Zu jeder kontextfreien Grammatik G kann man effektiv eine äquivalente Grammatik G' konstruieren, sodass G' in (erweiterter) Greibach NF ist.

[8] Sheila Greibach, *1939

Erweiterte Greibach Normalform - Beispiele

$L_1 = \left\{0^{2n}1^{4k}0^{2n} \mid n, k \geq 1\right\}$

Grammatik für L_1 in erweiterter Greibach Normalform:
$G_1 = \langle\{S, X\}, \{0, 1\}, P_1, S\rangle$ mit
$P_1 = \{S \to 0^2S0^2, S \to 0^2X0^2, X \to 1^4X, X \to 1^4\}$

Linksableitungen:
$S \Rightarrow^{n-1} 0^{2(n-1)}S0^{2(n-1)} \Rightarrow 0^{2n}X0^{2n} \Rightarrow^{k-1} 0^{2n}1^{4(k-1)}X0^{2n} \Rightarrow$
$0^{2n}1^{4k}0^{2n} \qquad \forall n, k \geq 1$

$L_2 = \{a^{n-4}b^k a^{5n} \mid n > 4, k \geq 3\}$

Grammatik für L_2 in erweiterter Greibach Normalform:
$G_2 = \langle\{S, A, B\}, \{a, b\}, P_2, S\rangle$ mit
$P_2 = \{S \to aAa^{25}, A \to aAa^5 \mid b^2B, B \to bB \mid b\}$

Eigenschaften kontextfreier Sprachen

Abschlusseigenschaften

Kontextfreie Sprachen sind **abgeschlossen** unter

- $\cup$, $\cdot$, *,
- Homomorphismen,
- Schnitt mit regulären Sprachen

Kontextfreie Sprachen sind **nicht abgeschlossen** unter

- Durchschnitt und Komplement.

Abschlusseigenschaften kontextfreier Sprachen

Kontextfreie Sprachen sind unter $\cup$, $\cdot$, * abgeschlossen.

Seien G_1, G_2 kontextfreie Grammatiken mit $N_1 \cap N_2 = \{\}$ und $S \notin N_1 \cup N_2$:

$$G_1 = \langle N_1, T_1, P_1, S_1 \rangle, \quad G_2 = \langle N_2, T_2, P_2, S_2 \rangle$$

Dann gilt:

$$G_{union} = \langle N_1 \cup N_2 \cup \{S\}, T_1 \cup T_2, \{S \to S_1 \mid S_2\} \cup P_1 \cup P_2, S \rangle$$
$$G_{cat} = \langle N_1 \cup N_2 \cup \{S\}, T_1 \cup T_2, \{S \to S_1 \cdot S_2\} \cup P_1 \cup P_2, S \rangle$$
$$G_{star} = \langle N_1 \cup \{S\}, T_1, \{S \to \varepsilon, S \to S_1 S\} \cup P_1, S \rangle$$

Sowie

$\mathcal{L}(G_{union})$, $\mathcal{L}(G_{cat})$, $\mathcal{L}(G_{star})$ kontextfrei

□

WKA revisited: Dyck[9] Sprachen

Dyck Sprachen

Sei D_n über $\Gamma_n = \{(_1,)_1, ..., (_n,)_n\}$ die kleinste Menge, für die gilt:

- $\varepsilon \in D_n$.
- Ist $v \in D_n$, so ist auch $(_i\, v)_i \in D_n, 1 \leq i \leq n$.
- Ist $v_1, v_2 \in D_n$, so ist auch $v_1 v_2 \in D_n$.

[9] Walther von Dyck, 1856-1934

Satz von Chomsky-Schützenberger[10]

Jede kontextfreie Sprache ist homomorphes Bild des Durchschnitts einer regulären Sprache mit einer Dyck-Sprache:

Satz von Chomsky-Schützenberger

Eine Sprache L über Σ ist genau dann kontextfrei, wenn zu einem $n \geq 0$ ein Homomorphismus $h : \Gamma_n^* \longrightarrow \Sigma^*$ so existiert, dass

$$L = h(D_n \cap R),$$

wobei R eine reguläre Sprache über Γ_n bezeichnet.

$L = \{a^n b^n \mid n \geq 0\}$ ist kontextfrei, da

$$L = h(D_1 \cap \{(\}^*\{)\}^*)$$

wobei $h : \{(,)\}^* \longrightarrow \{a, b\}^*$ mit $h(() = a, \quad h()) = b$

[10] Marcel-Paul Schützenberger, 1920 - 1996

Abschlusseigenschaften kontextfreier Sprachen

Kontextfreie Sprachen sind **nicht** abgeschlossen unter $\cap$.

$$L_1 = \{a^m b^m c^n \mid m, n \geq 0\} \qquad L_2 = \{a^m b^n c^n \mid m, n \geq 0\}$$

L_1, L_2 sind kontextfrei:

$$\begin{array}{lcl} S_1 & \to & S_1\, c \mid A \\ A & \to & a\, A\, b \mid \varepsilon \end{array} \qquad \begin{array}{lcl} S_2 & \to & a\, S_2 \mid B \\ B & \to & b\, B\, c \mid \varepsilon \end{array}$$

Nun gilt

$$L = L_1 \cap L_2 = \{a^m b^m c^m \mid m \geq 0\}$$

L ist aber **nicht** kontextfrei!

□

Abschlusseigenschaften kontextfreier Sprachen

Kontextfreie Sprachen sind **nicht** abgeschlossen unter Komplement

$$L_1 \cap L_2 = \overline{\overline{L_1} \cup \overline{L_2}}$$

Wir haben gesehen, dass kontextfreie Sprachen unter $\cup$ abgeschlossen sind. Wären diese auch unter Komplement abgeschlossen, so – wegen obiger Beziehung – auch unter Durchschnitt. Wir haben aber gezeigt, dass dies nicht der Fall ist.

□

Entscheidungsprobleme für kontextfreie Sprachen

Entscheidbare Probleme:

- Ist die k.f. Sprache L leer/endlich/unendlich?
- Liegt ein Wort w in der k.f. Sprache L? (Wort-Problem)

Unentscheidbare Probleme:

- Gilt $L = \Sigma^*$?
- $L_1 = L_2$ (Äquivalenzproblem)?
- $L_1 \subseteq L_2$?
- $L_1 \cap L_2 = \{\}$?
- Ist L regulär?
- Ist $L_1 \cap L_2$ bzw. $\Sigma^* - L$ kontextfrei?

Pumping Lemma für kontextfreie Sprachen

Pumping Lemma für kontextfreie Sprachen

Sei L eine unendliche kontextfreie Sprache. Dann existiert eine (nur von L abhängige) Schranke $m > 0$ so, dass für jedes Wort $w \in L$ mit $|w| \geq m$ Wörter u, v, x, y, z so existieren, dass

$$w = uvxyz \qquad \text{mit } |vxy| \leq m \text{ und } |vy| \geq 1$$

sowie

$$w_i = uv^i xy^i z \in L \qquad \text{für alle } i \geq 0.$$

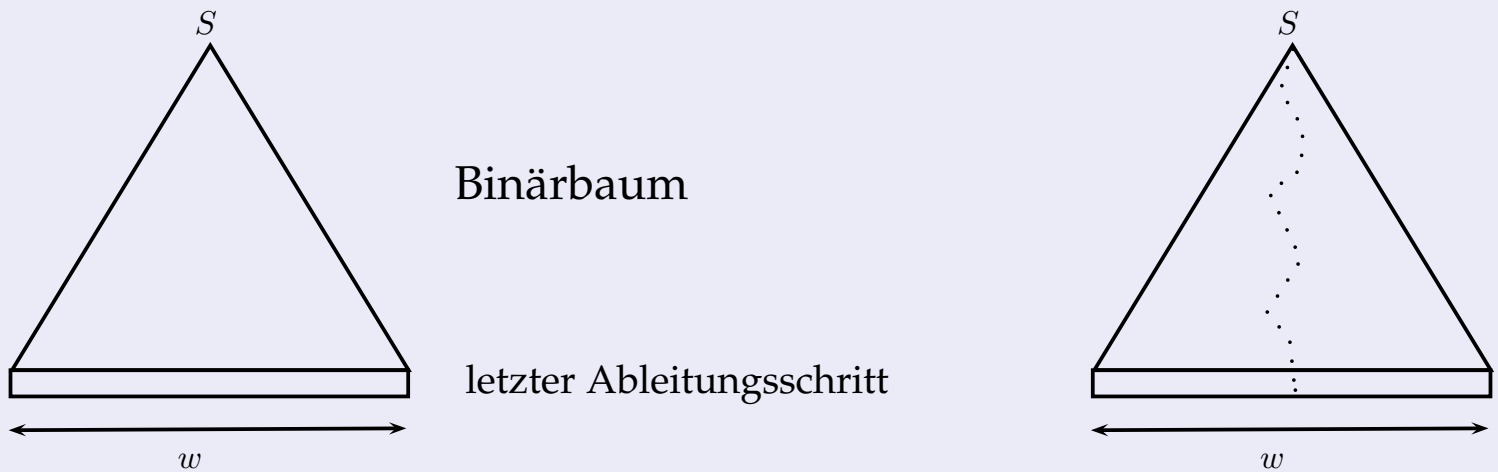
Beweis Idee:

Während man das Pumping Lemma für reguläre Sprachen mit der Anzahl der Zustände entlang eines Pfades in einem DEA begründet, kann man hier in ähnlicher Weise mit der Anzahl der Nonterminale entlang von Pfaden in den binären Ableitungsbäumen einer kontextfreien Grammatik (in Chomsky Normalform) argumentieren.

Pumping Lemma für kontextfreie Sprachen - Beweis

Beweis

Sei G eine Grammatik in Chomsky Normalform mit k Nonterminalen. Wähle $m = 2^k$. Betrachte nun den Ableitungsbaum eines beliebigen Wortes $w \in L(G)$ mit $|w| \geq m$. Dieser ist (bis auf den letzten Ableitungsschritt) ein Binärbaum:

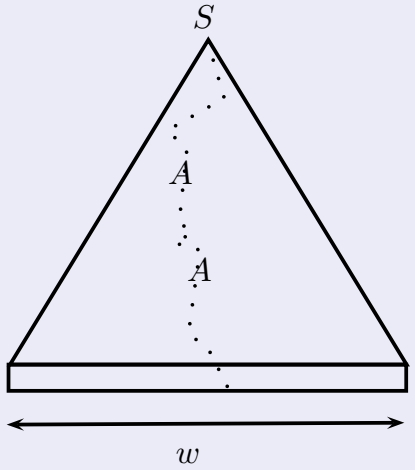


Dieser Binärbaum hat $\geq 2^k$ Blätter. Daher muss es mindestens einen Pfad der Länge $\geq k$ geben. Wir halten einen solchen Pfad fest.

Pumping Lemma für kontextfreie Sprachen - Beweis ctd.

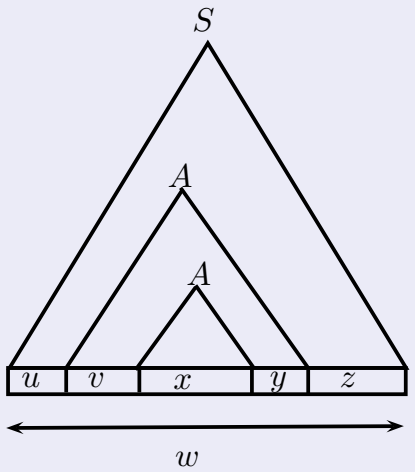
Beweis ctd.

Einschließlich des Startsymbols befinden sich auf diesem Pfad $\geq k+1$ Nonterminale. Da die Grammatik G nur k verschiedene Nonterminale besitzt, muss mindestens eines davon doppelt vorkommen (Schubfachprinzip!).



Wir betrachten nun die Teilwörter, die aus den beiden Nonterminalen A abgeleitet werden können. Diese induzieren eine Zerlegung von w in die Teilwörter $uvxyz$.

Da das obere Nonterminal A mittels einer Produktion der Form $A \to BC$ weiter abgeleitet wird, ist (sind) v oder y (oder beide) nicht leer.

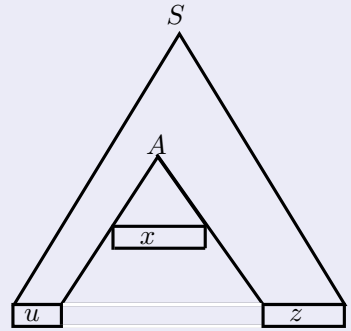


Pumping Lemma für kontextfreie Sprachen - Beweis ctd.

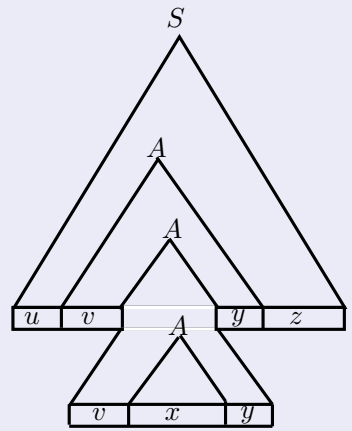
Beweis ctd.

Der Ableitungsbaum für w kann auf verschiedene Arten modifiziert werden:

Beispielsweise kann an das obere A der Ableitungsbaum des unteren A gehängt werden.
Wir erhalten damit eine Ableitung von $uxz = uv^0xy^0z$. Das heißt, $uv^0xy^0z \in L(G)$.



Man kann auch an das untere A den Teilbaum anhängen, der am oberen A beginnt. Dies ergibt eine Ableitung von $uvvxyyz = uv^2xy^2z$.



Allgemein gilt, dass für jedes $i \geq 0$ $uv^ixy^iz \in L(G)$. □

Pumping Lemma für k.f. Sprachen - Beispiel

$L = \{a^n b^n c^n \mid n \geq 0\}$ ist nicht kontextfrei

Beweis: indirekt. Angenommen L ist kontextfrei. Sei dann m die Konstante des Pumping Lemmas. Wähle z.B.

$$w = a^m b^m c^m$$

Für Wörter $w \in L$ gilt: $|w|_a = |w|_b = |w|_c$.

Da $|w| = 3m > m$ kann w zerlegt werden in $w = uvxyz$ mit $|vxy| \leq m$.

Daraus folgt, dass vxy nicht gleichzeitig Symbole a und Symbole c enthalten kann, d.h., entweder ist $|vxy|_a = 0$ oder $|vxy|_c = 0$.

(In der Tat liegen die Symbole a und c zu weit auseinander.)

Pumping Lemma für k.f. Sprachen - Beispiel ctd.

$$w = a^m b^m c^m, \quad w = uvxyz, \quad |vxy| \leq m, \quad |vy| \geq 1.$$

Fallunterscheidung:

Fall (a): $|vxy|_c = 0$.

Wegen $|vy| \geq 1$ gilt $|vy|_a + |vy|_b > 0$. Wähle $i = 0$, d.h.,

$$w_0 = uv^0xy^0z = uxz$$

(Nach dem Pumping Lemma müsste $w_0 \in L$ gelten.)

Für uxz gilt nun

$|uxz|_c = |uvxyz|_c = |w|_c$, aber $|uxz|_a < |w|_a$ oder $|uxz|_b < |w|_b$.

Das heißt, es gilt *nicht*

$|uxz|_a = |uxz|_b = |uxz|_c$ und daher $uxz \notin L$. **Widerspruch!**

Pumping Lemma für k.f. Sprachen - Beispiel ctd.

$$w = a^m b^m c^m, \quad w = uvxyz, \quad |vxy| \leq m, \quad |vy| \geq 1.$$

Fallunterscheidung ctd.:

Fall (b): $|vxy|_c > 0$ und daher $|vxy|_a = 0$.

Wegen $|vy| \geq 1$ gilt $|vy|_b + |vy|_c > 0$.

Für $w_0 = uxz$ gilt nun aber
$|uxz|_a = |uvxyz|_a = |w|_a$, jedoch $|uxz|_b < |w|_b$ oder $|uxz|_c < |w|_c$.
Das heißt, es gilt *nicht*
$|uxz|_a = |uxz|_b = |uxz|_c$ und daher $uxz \notin L$. **Widerspruch!**

Wir haben alle möglichen Zerlegungen von z untersucht, aber in jedem Fall einen Widerspruch erhalten! Somit kann L nicht kontextfrei sein. □

Korollar zum P.L. für kontextfreie Sprachen

Folgerung [Korollar zum P.L. für kontextfreie Sprachen]

Sei $L \subseteq \{a\}^*$, sodass $L = \{a^{f(n)} \mid n \geq 0\}$ für eine streng monoton wachsende Funktion f in den natürlichen Zahlen.
Gibt es für jede natürliche Zahl k eine natürliche Zahl $n(k)$, sodass

$$f(n(k+1)) - f(n(k)) \geq k,$$

dann kann L nicht kontextfrei sein.

Nach obigem Satz können die Sprachen

$$\{a^p \mid p \text{ ist eine Primzahl}\} \quad \text{und}$$
$$\{a^{2^n} \mid n \geq 0\}$$

nicht kontextfrei sein.

Kontextfreie Sprachen über einem einelementigen Alphabet

Bemerkung: Für kontextfreie Sprachen über einem einelementigen Alphabet gilt jedoch der folgende Satz:

Jede kontextfreie Sprache über einem einelementigen Alphabet ist regulär.

Jenseits der Kontextfreiheit

Formale Sprachen

$\{a^n b^n c^n \mid n \geq 0\}$ ist nicht kontextfrei.

Programmiersprachen

Typen- und Deklarationsbedingungen sind nicht oder nur schwer kontextfrei darstellbar.

Natürliche Sprachen

Schweizer Dialekt:

Jan säit das *mer d'chind em Hans es huus haend wele laa hälfe aastriiche.*

Hochdeutsch:
Jan sagt, dass *wir die Kinder dem Hans helfen lassen wollten, das Haus anzustreichen.*

Monotone Grammatiken

Monotone Grammatik

Eine Grammatik heißt *monoton*, wenn für alle Produktionen $\alpha \to \beta$ die Länge von α kleiner gleich der Länge von β ist ($\alpha \neq \varepsilon$).
Kommt das Startsymbol S auf keiner rechten Seite vor, ist auch $S \to \varepsilon$ zugelassen.

$L = \{a^n b^n c^n \mid n \geq 0\}$

$G = (\{S, T, C\}, \{a, b, c\}, P, S)$

wobei P folgende Produktionen enthält:

$$\begin{array}{lcl} S & \to & \varepsilon \mid abc \mid a\,T\,bc \\ T & \to & a\,T\,b\,C \mid ab\,C \end{array} \qquad \begin{array}{lcl} C\,b & \to & b\,C \\ C\,c & \to & cc \end{array}$$

Es gilt: $\mathcal{L}(G) = L$.

Kontextsensitive Grammatiken

Kontextsensitive Grammatik

Eine Grammatik heißt *kontextsensitiv*, wenn alle Produktionen die Form $uAv \to u\beta v$ besitzen, wobei $u, v \in (N \cup T)^*$, $A \in N$ und $\beta \in (N \cup T)^+$.
Kommt Startvariable S auf keiner rechten Seite vor, ist auch $S \to \varepsilon$ zugelassen.

Für jede Sprache, die von einer monotonen Grammatik erzeugt wird, gibt es auch eine kontextsensitive Grammatik, und umgekehrt.

Kontextsensitive Grammatiken: Beispiel

$L = \{a^n b^n c^n \mid n \geq 0\}$

$G = (\{S, T, B, C, X, Y\}, \{a, b, c\}, P, S)$

wobei P folgende Produktionen enthält:

$$\begin{array}{rcl} S & \to & \varepsilon \mid abc \mid aTBc \\ T & \to & aTBC \mid abC \\ B & \to & b \\ Cc & \to & cc \end{array} \qquad \begin{array}{rcl} CB & \to & CY \\ CY & \to & XY \\ XY & \to & XC \\ XC & \to & BC \end{array}$$

Es gilt: $\mathcal{L}(G) = L$.

Sprachfamilien

Sprachfamilien

Sei $i \in \{0, 1, 2, 3\}$ und Σ ein Alphabet. Dann wird die Menge aller formalen Sprachen $L \subseteq \Sigma^*$, die von einer Grammatik vom Typ i erzeugt werden können, mit $\mathcal{L}_i(\Sigma)$ bezeichnet. Die Familie der formalen Sprachen, die von einer Typ-i-Grammatik erzeugt werden können, bezeichnen wir mit $\mathcal{L}_i$.

Normalformen für Typ-i-Grammatiken

Typ-3:	$A \to b$,	$A \to bC$,		$b \in T$
Typ-2:	$A \to b$,	$A \to BC$,		$b \in T$
Typ-1:	$A \to b$,	$A \to BC$,	$AD \to BC$,	$b \in T$
Typ-0:	$A \to b$,	$A \to BC$,	$AD \to BC$,	$b \in T \cup \{\varepsilon\}$

Bei den Typen 3, 2, und 1 wird noch die Produktion $S \to \varepsilon$ erlaubt, sofern das Startsymbol S nicht auf der rechten Seite einer Produktion vorkommt.

Wie man aus diesen Normalformen für die Typ-i Grammatiken sofort erkennt, gilt

$$\mathcal{L}_3 \subseteq \mathcal{L}_2 \subseteq \mathcal{L}_1.$$

Rekursive Sprachen

Rekursive (entscheidbare) Sprachen

Eine Sprache $L \in \mathcal{L}_0(\Sigma)$ heißt *rekursiv* (*entscheidbar*), wenn auch das Komplement der Sprache $\overline{L} = \Sigma^* - L$, eine rekursiv aufzählbare Sprache ist, d.h., $\overline{L} \in \mathcal{L}_0(\Sigma)$. Die Menge der rekursiven Sprachen aus $\mathcal{L}_0(\Sigma)$ wird mit $\mathcal{L}_{rec}(\Sigma)$ bezeichnet, die Familie der rekursiven Sprachen mit $\mathcal{L}_{rec}$.

Wortproblem für rekursive Sprachen

Wortproblem

Das Problem $w \in L$ ist für rekursive Sprachen L entscheidbar.

Beweis. [Idee] Einfacher Entscheidungsalgorithmus:
gegeben Grammatik G für eine Sprache L sowie
Grammatik G' für das Komplement von L, d.h. $\overline{L}$
Berechne schrittweise alle möglichen Satzformen für G und G', die in $n = 1, 2, \ldots$ Schritten ableitbar sind. Da nun das Wort w entweder in $\mathcal{L}(G)$ oder $\mathcal{L}(G')$ liegt, muss es ein n geben, sodass w in n Schritten entweder in G oder in n Schritten in G' ableitbar ist, d.h., in den in n Schritten ableitbaren Satzformen vorkommt. □

Chomsky-Hierarchie

Sprachfamilie	Grammatiktyp	Maschinenmodell
$\mathcal{L}_0$ (rekursiv aufzählb.)	unbeschränkt	Turingmaschine (= EA + RAM)
$\mathcal{L}_1$ (kontextsensitiv)	kontextsensitiv monoton	Linear beschränkter Aut. (= EA + beschr.RAM)
$\mathcal{L}_2$ (kontextfrei)	kontextfrei	Kellerautomat (= EA + Stack)
$\mathcal{L}_3$ (regulär)	regulär	endlicher Automat (EA)

Chomsky Hierarchie

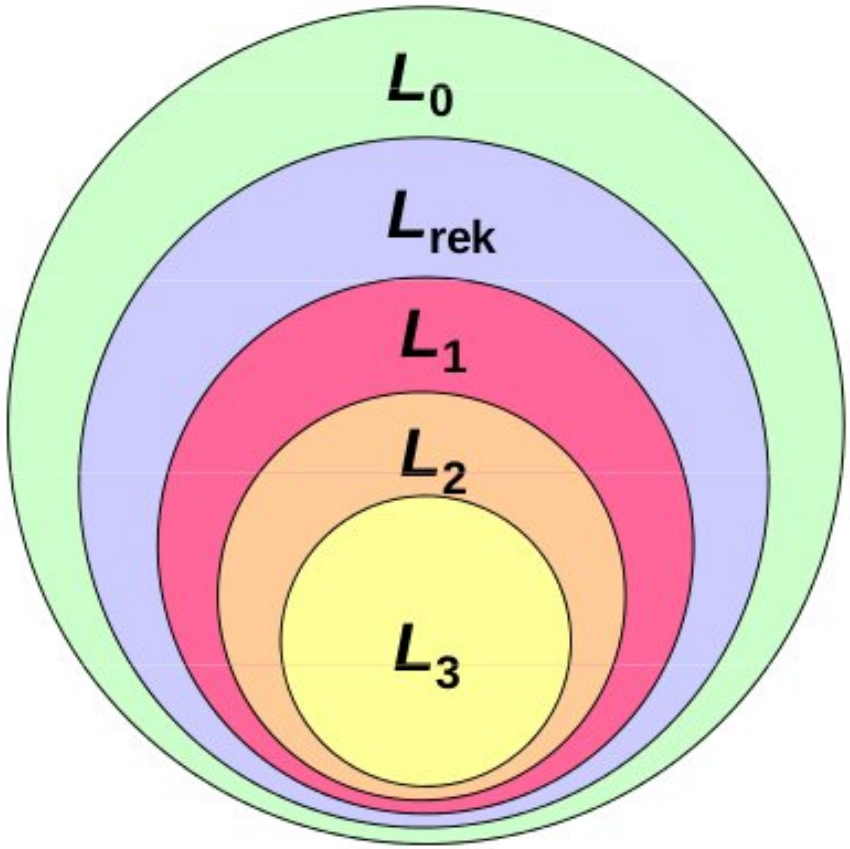
$$\mathcal{L}_3 \subsetneqq \mathcal{L}_2 \subsetneqq \mathcal{L}_1 \subsetneqq \mathcal{L}_{rec} \subsetneqq \mathcal{L}_0.$$

Chomsky-Hierarchie

Chomsky Hierarchie

$$\mathcal{L}_3 \subset \mathcal{L}_2 \subset \mathcal{L}_1 \subset \mathcal{L}_{rec} \subset \mathcal{L}_0.$$

nicht rekursiv aufzählbare Sprachen



Abschlusseigenschaften von Sprachfamilien

Abschluss unter regulären Operationen

Alle Sprachfamilien $\mathcal{L}_i$, $i \in \{0, 1, 2, 3\}$, der Chomsky-Hierarchie sind gegenüber den Operationen Vereinigung, Konkatenation und Kleene-Stern abgeschlossen.

Abschlusseigenschaften: Homomorphismen

Seien Σ und Γ zwei Alphabete. **Homomorphismus**: $h\colon \Sigma \to \Gamma^*$
Induktive Erweiterung auf Wörter über Σ:

1. $h(\varepsilon) = \varepsilon$;
2. $h(wa) = h(w)h(a)$ für alle $w \in \Sigma^*$ und $a \in \Sigma$.

Anwendung von h auf jedes Wort der Sprache L:

$$h(L) = \{h(w) \mid w \in L\}$$

Ein Homomorphismus h heißt ε-**frei**, wenn $h(a) \neq \varepsilon$ für alle $a \in \Sigma$.

Abschluss unter Homomorphismen

Die Sprachfamilien $\mathcal{L}_i$, $i \in \{0, 2, 3\}$, sind gegenüber beliebigen Homomorphismen abgeschlossen; die Familie der monotonen (kontext-sensitiven) Sprachen ist nur gegenüber ε-freien Homomorphismen abgeschlossen.

Homomorphismen: Beispiel

$L = \{a^{m^4} b^{mn} c^{mn} a^n d^{mn} e^m \mid n \geq 1\}$ nicht kontextfrei

Wir zeigen indirekt, dass L nicht kontextfrei sein kann:

Sei h der Homomorphismus mit

$$h : \{a, b, c, d, e\}^* \to \{a, b, c\}^* \text{ sowie}$$

$$h(a) = \varepsilon, \quad h(b) = a, \quad h(c) = b, \quad h(d) = c, \text{ und } h(e) = \varepsilon,$$

d.h., $h(L) = \{a^{mn} b^{mn} c^{mn} \mid n \geq 1\}$.

Die Familie der kontextfreien Sprachen ist gegenüber beliebigen Homomorphismen abgeschlossen, d.h. auch $h(L)$ müsste daher kontextfrei sein. $h(L)$ ist aber nicht kontextfrei, demnach kann auch L keine kontextfreie Sprache sein.

gsm-Abbildungen

gsm (generalized sequential machine): endlicher Automat mit Ausgabe

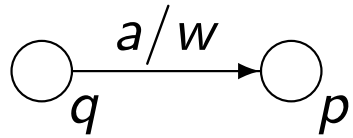
Eine *gsm* ist ein Sechstupel $M = \langle Q, \Sigma, \Gamma, \delta, q_0, F \rangle$, wobei

Q Zustände, Σ Eingabealphabet, Γ Ausgabealphabet,

$\delta : Q \times \Sigma \to Q \times \Gamma^*$.

q_0 Startzustand, F Endzustände

Bedeutung von $\delta(q, a) = (p, w)$: Beim Übergang von q nach p wird Symbol a gelesen und Wort w ausgegeben:



gsm-Abbildungen

gsm-Abbildung $f_M : \Sigma^* \to \mathcal{P}(\Gamma^*)$
$f_M(w)$ definiert durch alle Ausgabewörter v, die sich bei Analyse des Eingabewortes w auf einem Pfad vom Startknoten q_0 zu einem Endknoten aus F ergeben.
Erweiterung auf Sprachen $L \subseteq \Sigma^*$:

$$f_M(L) := \{v \in \Gamma^* \mid v \in f_M(w) \text{ für ein } w \in L\}$$

M und f_M heißen ε-*frei*, wenn $\delta : Q \times \Sigma \to Q \times \Gamma^+$.
M und f_M heißen *deterministisch*, wenn für jedes Paar (q, a) höchstens ein $(q, a; p, v) \in \delta$.

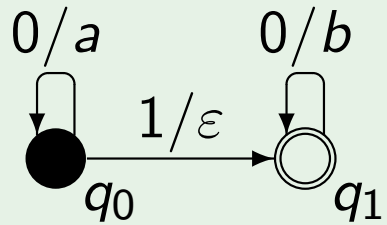
gsm-Abbildungen: Beispiel

gsm M bildet $\{0^n 1 0^n \mid n \geq 0\}$ auf $\{a^n b^n \mid n \geq 0\}$ ab:

$$M = (\{q_0, q_1\}, \{0, 1\}, \{a, b\}, \delta, q_0, \{q_1\})$$

mit

$\delta(q_0, 0) = (q_0, a)$
$\delta(q_0, 1) = (q_1, \varepsilon)$
$\delta(q_1, 0) = (q_1, b)$



Homomorphismen und gsm-Abbildungen

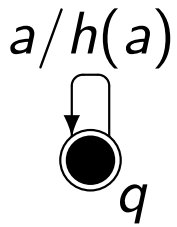
Jeder Homomorphismus ist eine sehr einfache gsm-Abbildung, die nur einen einzigen Zustand benötigt:

Für einen gegebenen Homomorphismus h ist die zugehörige gsm M_h folgendermaßen definiert:

$$M_h = (\{q\}, \Sigma, \Gamma, \delta, q, \{q\})$$

mit

$$\delta(q, a) = (q, h(a)) \quad \text{für alle } a \in \Sigma$$



Abschlusseigenschaften: gsm-Abbildungen

Abschluss unter gsm-Abbildungen

Für $i \in \{0, 2, 3\}$ sind die Familien $\mathcal{L}_i$ gegenüber beliebigen gsm-Abbildungen und damit auch gegenüber beliebigen Homomorphismen abgeschlossen; die Familie der monotonen (kontext-sensitiven) Sprachen ist nur gegenüber ε-freien gsm-Abbildungen und damit auch gegenüber ε-freien Homomorphismen abgeschlossen.

Abschlusseigenschaften von Sprachfamilien

	$\mathcal{L}_3$	$\mathcal{L}_2$	$\mathcal{L}_1$	$\mathcal{L}_0$
Vereinigung	ja	ja	ja	ja
Konkatenation	ja	ja	ja	ja
Kleenescher Stern	ja	ja	ja	ja
Komplement	ja	**nein**	ja	**nein**
Durchschnitt	ja	**nein**	ja	ja
Durchschnitt mit reg. Mengen	ja	ja	ja	ja
Homomorphismen	ja	ja	nein	ja
ε-freie Homomorphismen	ja	ja	ja	ja
gsm-Abbildungen	ja	ja	nein	ja
ε-freie *gsm*-Abbildungen	ja	ja	ja	ja